Projeto Integrado em Redes e Sistemas Informático

ESTGA

CURSO SUPERIOR

Redes e Sistemas Informáticos

ESCOLA

Escola Superior de Tecnologia e Gestão de Águeda

FORMANDOS

Tomás Calçada - 127069

Rodrigo Oliveira – 126843

Ruben Santiago – 126853

João Almeida – 127140

Rafael Seixas – 120947

ANO LETIVO

2025/2026

Índice:

Introdução 2
Introduction 3
ORÇAMENTO FINAL DO PROJETO 5
Equipamentos de Rede: 6
Tabela de Endereçamento IP: 6
Estruturas de Rede dos Sites 7
Vlans e Sub-Interfaces: 10
Configuração Vlans/Sub-Interfaces: 11
OSPF: 15
Configurações OSPF (Routers de distribuição): 16
NAT e Access-Lists: 17
Configuração do NAT e suas Access-Lists: 18
Configurações DHCP e DNS 21
DHCP 22
Configuração DHCP 22
Configuração do ficheiro dhcpd.conf: 22
DNS Interno no Windows Server 26
Dns Externo 27
Implementação da VPN Site-to-Site (IPsec) 27
Configurações dos Tuneis IPSEC: 28
Access Point (AP): 32
Configurações do AP: 33
Configuração e Implementação VPN: 35
Configuração da VPN de Acesso Remoto (OpenVPN) 36
Configurações Chave: 36
Comandos Utilizados: 36
Testes de Validação: 36
NAT e Port-Forwarding no Router Cisco 36
Validação: 37
Servidor de Mail (IredMAIL): 37
Servidor WEB: 40
Configuração VOIP – Asterisk: 41
Configuração do Kamailio: 42

Ficheiro de configuração /etc/kamailio/kamailio.cfg:....42
Módulos Carregados:....43
Parâmetros dos Módulos:....43
Lógica de Routing SIP:....44
Proteção contra Loops SIP:....44
Registo de Utilizadores (REGISTER):....44
Estabelecimento de Chamadas (INVITE):....45
Base de Dados MariaDB:....45
Configuração de DNS:....46
Registos Diretos (A):....46
Registos Inversos (PTR):....46
Registos SRV para SIP:....46
Configuração de NAT no Router Cisco de Águeda....47
Encaminhamento da Porta SIP:....47
Encaminhamento de RTP (Áudio):....47
Testes e Validação....47
FreeRadius:....48
Configuração FreeRadius:....48
Active Directory:....49
Arquitetura da Solução:....50
Clientes finais:....50
Implementação do Active Directory:....50
Integração do FreeRADIUS com o Active Directory:....51
Configuração do Access Point Cisco:....51
SSID de Convidados:....52
Validação e Testes:....53
Backups Centralizados....53
Automação de Backups e Gestão de Redundância....54
Metodologia de Captura (Protocolo Telnet)....54
Lógica de Versionamento Local (Cleanup)....54
Redundância em Nuvem (Integração Rclone)....54
Monitorização e Diagnóstico....54
Discord Bot-Ticketing....57

Infraestrutura em Docker 58
NGINX 60
Zabbix 61
Monitorização da Infraestrutura com Zabbix: 61
Descoberta Automática de Dispositivos: 62
Hosts Monitorizados: 62
Monitorização via SNMP: 63
Painéis de Controlo e Visão Global: 64
Integração Zabbix–Discord: 65
Modelo de Dados: Item do Tipo Trapper: 66
Lógica de Disparo: Trigger e Estados de Incidente: 66
Classificação de Severidade: 66
Configuração do Host e da Trigger: 66
Integração de Saída: Webhook e Tipos de Media: 67
Grafana 67
Integração do Grafana com o Zabbix e o Loki 68
Integração do Grafana com o Zabbix 68
Integração do Grafana com o Loki 68
Endpoint do Grafana: 69
Bot Pyhton 70
Arquitetura Híbrida (Bot + Web Server): 71
Fluxo de Execução do Comando /incident: 71
Ambiente: 71
Resumo das Bibliotecas Utilizadas: 71
Armazena informações de claims de alertas 72
Dicionário para rastrear status dos incidentes 72
Estados disponíveis 72
Conclusão 76

Introdução

Este relatório apresenta um panorama detalhado do projeto integrado que foi desenvolvido no âmbito do curso de Redes e Sistemas Informáticos. O foco principal do projeto foi a análise, o planeamento e/ou a implementação de soluções de infraestrutura e comunicação para a empresa parceira, “Moliceiros da Ria”.

A "Moliceiros da Ria" atua no setor do turismo e tem uma forte presença no distrito de Aveiro. Sua estrutura organizacional é composta por equipas independentes que gerenciam clientes específicos, o que exige um ambiente de rede e sistemas que favoreça essa autonomia e uma comunicação eficaz.

As próximas seções deste relatório trarão uma análise minuciosa da arquitetura existente, os requisitos identificados e as propostas/soluções técnicas que foram desenvolvidas para otimizar a infraestrutura de TI da "Moliceiros da Ria", assegurando a coesão e a eficácia das suas operações em todas as suas localizações.

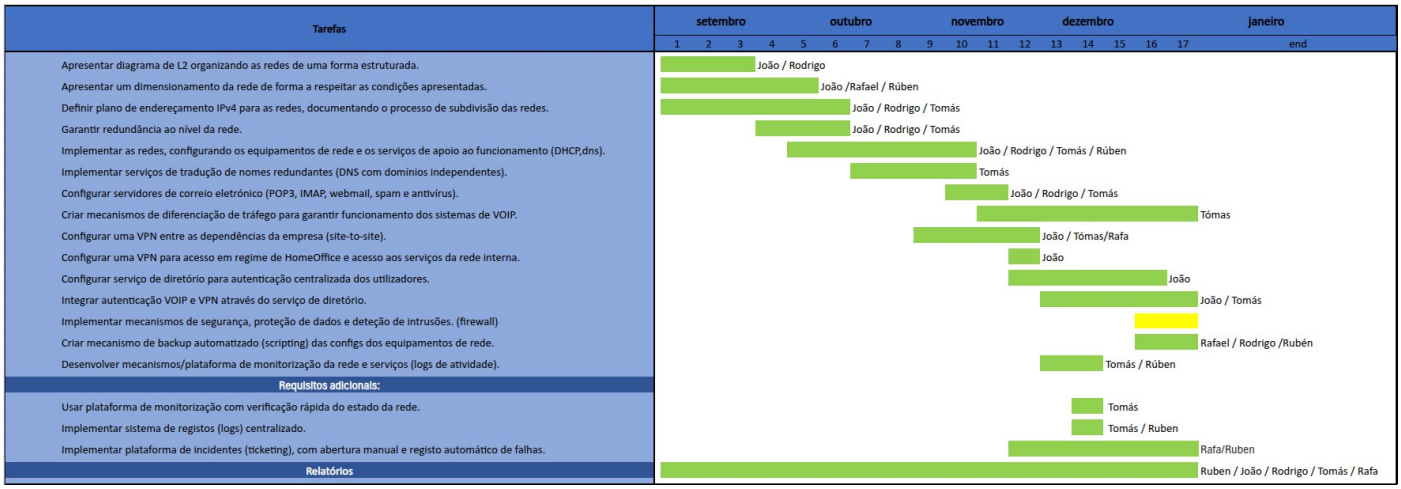
Introduction

This report presents a detailed overview of the integrated project that was developed as part of the Computer Networks and Systems course. The main focus of the project was the analysis, planning and implementation of infrastructure and communication solutions for the partner company, Moliceiros da Ria.

Moliceiros da Ria operates in the tourism sector and has a strong presence in the district of Aveiro. Its organisational structure consists of independent teams that manage specific clients, which requires a network and systems environment that promotes this autonomy and effective communication.

The following sections of this report will provide a detailed analysis of the existing architecture, the requirements identified, and the technical proposals/solutions that have been developed to optimise the IT infrastructure of Moliceiros da Ria, ensuring the cohesion and effectiveness of its operations across all its locations.

Diagrama de Gantt:



ORÇAMENTO FINAL DO PROJETO

Equipamentos de Rede:

Equipamento	Quantidade	Preço unitário (€)	Subtotal (€)
Cisco 1800 Series Router (NAT)	5	225	1125
Cisco 2800 Series Router	5	270	1350
Cisco Catalyst 2950 Series Switch	5	85	425
Cisco Aironet 1200 Series (WiFi)	6	150	900
Subtotal Rede			3800€

Voz sobre IP:

Equipamento	Quantidade	Preço unitário (€)	Subtotal (€)
Cisco SPA941 IP Phone	35	50	1750
Subtotal VoIP			1750€

Computadores e Servidores

Computadores:

Equipamento	Quantidade	Preço unitário (€)	Subtotal (€)
HP EliteDesk 800 G3 Mini (novo/NOS)	82	450	36900

Servidores:

Equipamento	Quantidade	Preço unitário (€)	Subtotal (€)
Servidor empresarial (HP/Dell, NOS)	4	2800	11200

Subtotal Computadores + Servidores = 62 000€

Mão de Obra (5 funcionarios	**5(funcionarios)*60(horas totais) * 6.50(a hora)**	**~2000**

RESUMO GERAL

Categoria	Total (€)
Rede	3800
VoIP	1750
Computadores e Servidores	48100
Mão de obra	~2000
TOTAL GERAL (sem IVA)	55650

Tabela de Endereçamento IP:

	VLAN	Nome	Subnet	Máscara decimal	Network ID	Gateway	Broadcast
Águeda - Sede	10	Administração	10.1.10.0/28	255.255.255.240	10.1.10.0	10.1.10.1	10.1.10.15
	20	Desenvolvimento	10.1.20.0/28	255.255.255.240	10.1.20.0	10.1.20.1	10.1.20.15
	15	Publicidade	10.1.15.0/28	255.255.255.240	10.1.15.0	10.1.15.1	10.1.15.15
	30	Guias_Animadores	10.1.30.0/28	255.255.255.240	10.1.30.0	10.1.30.1	10.1.30.15
	35	VOIP	10.1.35.0/28	255.255.255.240	10.1.35.0	10.1.35.1	10.1.35.15
	100	Guests	10.1.100.0/24	255.255.255.0	10.1.100.0	10.1.100.1	10.1.100.255
	200	Infra_Servidores	10.1.120.0/26	255.255.255.192	10.1.120.0	10.1.120.1	10.1.120.63
	300	Wi-fi	10.1.130.0/26	255.255.255.192	10.1.130.0	10.1.130.1	10.1.130.63
Aveiro - Polo 1	10	Administração	10.2.10.0/28	255.255.255.240	10.2.10.0	10.2.10.1	10.2.10.15
	30	Guias_Animadores	10.2.30.0/28	255.255.255.240	10.2.30.0	10.2.30.1	10.2.30.15
	5	Financeiro	10.2.5.0/28	255.255.255.240	10.2.5.0	10.2.5.1	10.2.5.15
	35	VoIP	10.2.35.0/28	255.255.255.240	10.2.35.0	10.2.35.1	10.2.35.15
	100	Guests	10.2.100.0/24	255.255.254.0	10.2.100.0	10.2.100.1	10.2.100.255
	200	Infra_Servidores	10.2.120.0/26	255.255.255.192	10.2.120.0	10.2.120.1	10.2.120.63
	300	Wi-fi	10.2.130.0/26	255.255.255.192	10.2.130.0	10.2.130.1	10.2.130.63
OAZ - Polo 2	20	Desenvolvimento	10.3.20.0/27	255.255.255.224	10.3.20.0	10.3.20.1	10.3.20.31
	25	Recursos_Humanos	10.3.25.0/28	255.255.255.240	10.3.25.0	10.3.25.1	10.3.25.15
	100	Guests	10.3.100.0/28	255.255.255.240	10.3.100.0	10.3.100.1	10.3.100.15
	200	Infra_Servidores	10.3.120.0/26	255.255.255.192	10.3.120.0	10.3.120.1	10.3.120.63
	300	Wi-fi	10.3.130.0/26	255.255.255.192	10.3.130.0	10.3.130.1	10.3.130.63

Primeiro IP utilizável	Último IP utilizável	Total hosts	Hosts utilizáveis	ESW1	ESW2	dominio	IP públicos
10.1.10.2	10.1.10.14	16	14	10.1.10.2	----------	agueda.office.moliceiro3.info	192.168.229.146
10.1.20.2	10.1.20.14	16	14	10.1.20.2	----------	agueda.office.moliceiro3.info	192.168.229.146
10.1.15.2	10.1.15.14	16	14	10.1.15.2	----------	agueda.office.moliceiro3.info	192.168.229.146
10.1.30.2	10.1.30.14	16	14	10.1.30.2	----------	agueda.office.moliceiro3.info	192.168.229.146
10.1.35.2	10.1.35.14	16	14	10.1.35.2	----------	agueda.office.moliceiro3.info	192.168.229.146
10.1.100.2	10.1.100.2550	256	255	10.1.100.2	----------	agueda.office.moliceiro3.info	192.168.229.146
10.1.120.2	10.1.120.62	64	62	10.1.120.2	----------	agueda.office.moliceiro3.info	192.168.229.146
10.1.130.2	10.1.130.62	64	62	10.1.130.2	----------	agueda.office.moliceiro3.info	192.168.229.146
10.2.10.2	10.2.10.14	16	14	10.2.10.3	10.2.10.3	aveiro.office.moliceiro3.info	192.168.229.17
10.2.30.2	10.2.30.14	16	14	10.2.30.3	10.2.30.3	aveiro.office.moliceiro3.info	192.168.229.17
10.2.5.2	10.2.5.14	16	14	10.2.5.3	10.2.5.3	aveiro.office.moliceiro3.info	192.168.229.17
10.2.35.2	10.2.35.14	16	14	10.2.35.3	10.2.35.3	aveiro.office.moliceiro3.info	192.168.229.17
10.2.100.2	10.2.100.254	256	254	10.2.100.3	10.2.100.3	aveiro.office.moliceiro3.info	192.168.229.17
10.2.120.2	10.2.120.62	64	62	10.2.120.3	10.2.120.3	aveiro.office.moliceiro3.info	192.168.229.17
10.2.130.2	10.2.130.62	64	62	10.2.130.3	10.2.130.3	aveiro.office.moliceiro3.info	192.168.229.17
10.3.20.2	10.3.20.30	32	30	10.3.20.2	10.3.20.3	azemeis.office.moliceiro3.info	192.168.229.19
10.3.25.2	10.3.25.14	16	14	10.3.25.2	10.3.25.3	azemeis.office.moliceiro3.info	192.168.229.19
10.3.100.2	10.3.100.14	16	14	10.3.100.2	10.3.100.3	azemeis.office.moliceiro3.info	192.168.229.19
10.3.120.2	10.3.120.62	64	62	10.3.120.2	10.3.120.3	azemeis.office.moliceiro3.info	192.168.229.19
10.3.130.2	10.3.130.62	64	62	10.3.130.2	10.3.130.3	azemeis.office.moliceiro3.info	192.168.229.19

Estruturas de Rede dos Sites

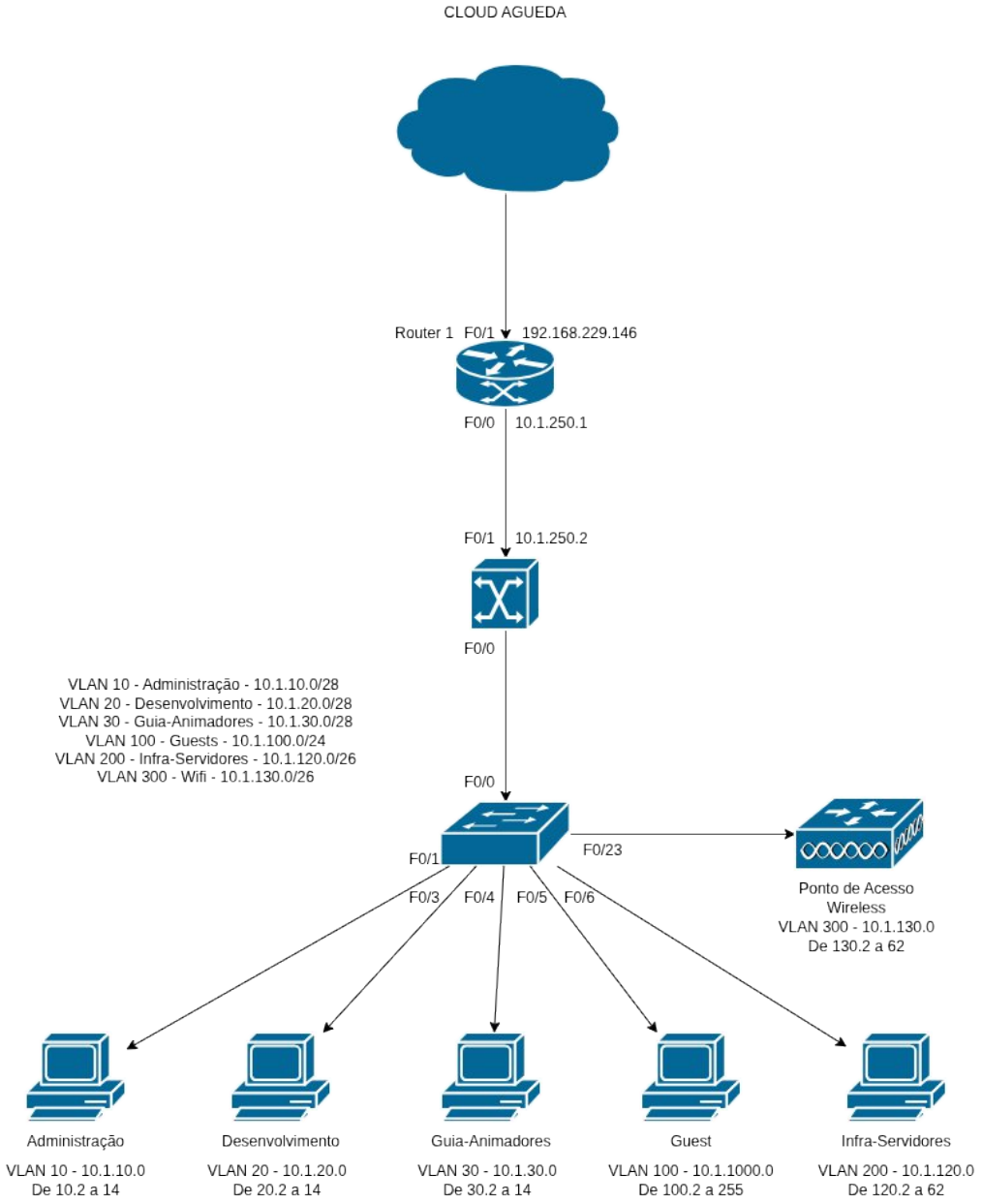


Figura 1 - Estrutura de Rede Site Águeda

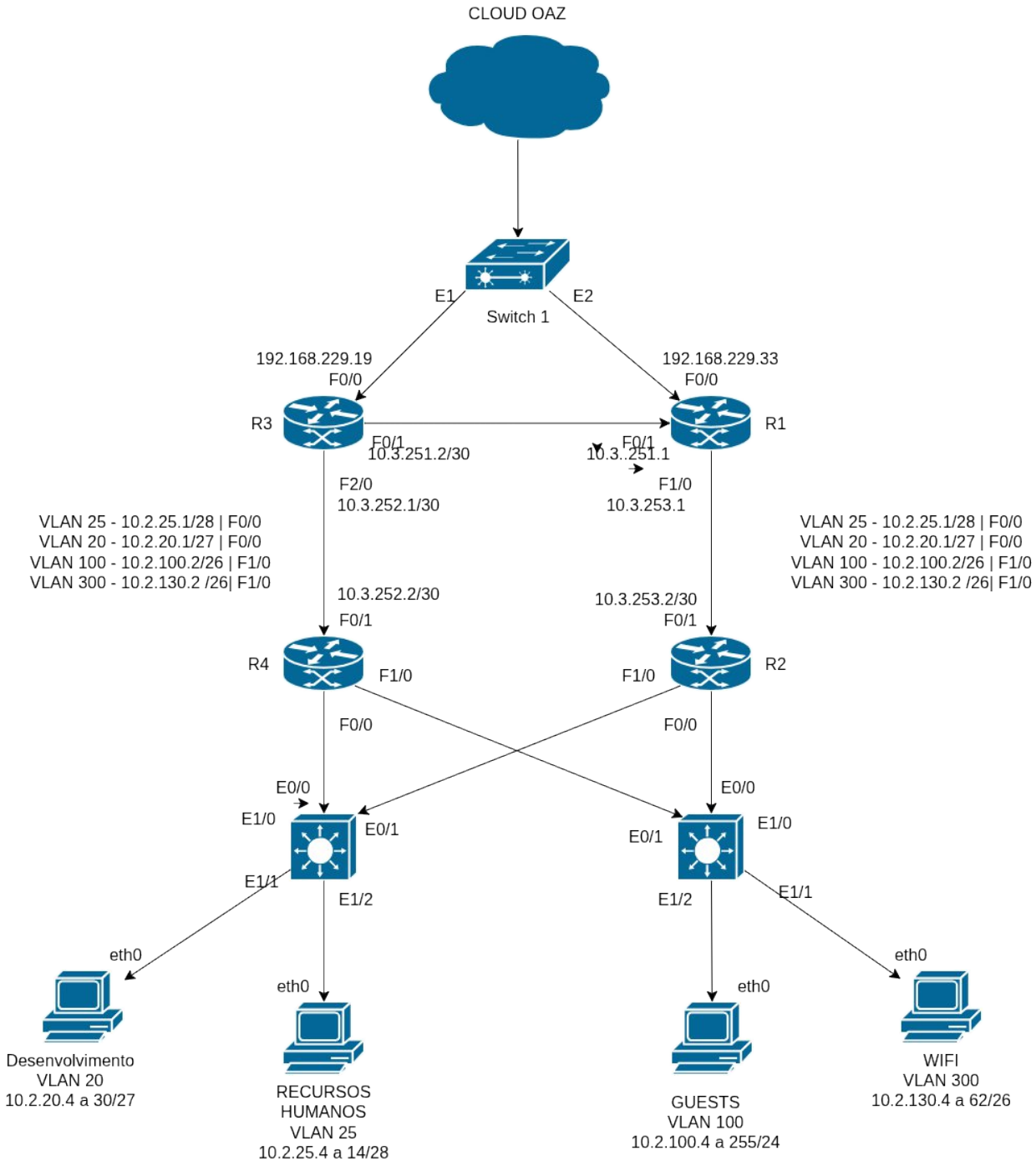


Figura 2 - Estrutura de Rede Site OAZ

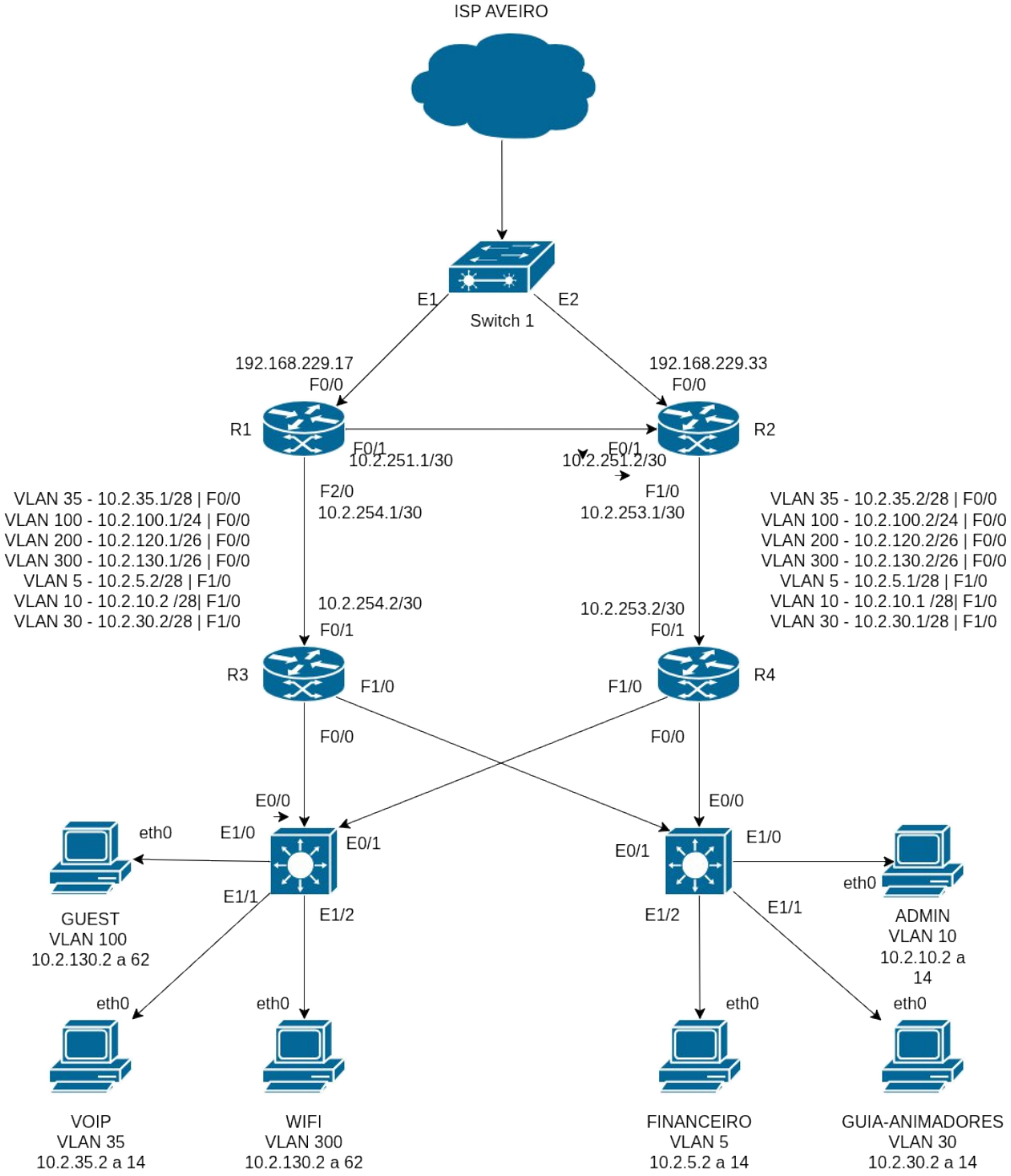


Figura 3 - Estrutura de Rede Aveiro

Vlans e Sub-Interfaces:

As VLANs (Virtual Local Area Networks) permitem dividir logicamente uma rede física em várias redes independentes, melhorando a organização, o desempenho e a segurança da comunicação. Dispositivos pertencentes a VLANs diferentes ficam isolados ao nível da camada 2, mesmo que estejam ligados ao mesmo equipamento de rede.

As sub-interfaces são utilizadas, normalmente em routers, para permitir a comunicação entre VLANs através de uma única interface física. Cada sub-interface é associada a uma VLAN específica e configurada com um endereço IP

Configuração Vlans/Sub-Interfaces:

Sede - Águeda
interface FastEthernet0/0 no shutdown interface FastEthernet0/0.10 encapsulation dot1Q 10 description VLAN10 Administracao ip address 10.1.10.1 255.255.255.240 interface FastEthernet0/0.20 encapsulation dot1Q 20 description VLAN20 Desenvolvimento ip address 10.1.20.1 255.255.255.240 interface FastEthernet0/0.15 encapsulation dot1Q 15 description VLAN15 Publicidade ip address 10.1.15.1 255.255.255.240 interface FastEthernet0/0.30

```
encapsulation dot1Q 30
description VLAN30 Guias_Animadores
ip address 10.1.30.1 255.255.255.240

interface FastEthernet0/0.100
encapsulation dot1Q 100
description VLAN100 Guests
ip address 10.1.100.1 255.255.255.0

interface FastEthernet0/0.200
encapsulation dot1Q 200
description VLAN200 Infra_Servidores
ip address 10.1.120.1 255.255.255.192

interface FastEthernet0/0.300
encapsulation dot1Q 300
description VLAN300 Wi-fi
ip address 10.1.130.1 255.255.255.192
```

Aveiro – Polo 1

```
Interface FastEthernet0/0
no shutdown

interface FastEthernet0/0.10
encapsulation dot1Q 10
description VLAN10 Administracao
ip address 10.2.10.1 255.255.255.240

interface FastEthernet0/0.5
encapsulation dot1Q 5
description VLAN5 Financeiro
ip address 10.2.11.1 255.255.255.240

interface FastEthernet0/0.30
encapsulation dot1Q 30
description VLAN30 Guias_Animadores
ip address 10.2.12.1 255.255.255.224

interface FastEthernet0/0.35
encapsulation dot1Q 35
description VLAN35 VoIP
ip address 10.2.13.1 255.255.255.192

interface FastEthernet0/0.200
encapsulation dot1Q 200
description VLAN200 Infra_Servidores
ip address 10.2.14.1 255.255.255.192

interface FastEthernet0/0.300
encapsulation dot1Q 300
description VLAN300 Wi-fi
ip address 10.2.15.1 255.255.255.192
```

```
interface FastEthernet0/0.100
encapsulation dot1Q 100
description VLAN100 Guests
ip address 10.2.20.1 255.255.254.0
```

OAZ – POLO 2

```
interface FastEthernet0/0
no shutdown

interface FastEthernet0/0.20
encapsulation dot1Q 20
description VLAN20 Desenvolvimento
ip address 10.3.20.1 255.255.255.224

interface FastEthernet0/0.25
encapsulation dot1Q 25
description VLAN25 Recursos_Humanos
ip address 10.3.25.1 255.255.255.240

interface FastEthernet0/0.100
encapsulation dot1Q 100
description VLAN100 Guests
ip address 10.3.100.1 255.255.255.0

interface FastEthernet0/0.200
encapsulation dot1Q 200
description VLAN200 Infra_Servidores
ip address 10.3.120.1 255.255.255.192

interface FastEthernet0/0.300
encapsulation dot1Q 300
description VLAN300 Wi-fi
ip address 10.3.130.1 255.255.255.192
```

OSPF:

O OSPF funciona através da partilha do estado das ligações entre os routers, permitindo que cada um construa uma visão completa da rede e calcule automaticamente as melhores rotas. Desta forma, o protocolo adapta-se rapidamente a alterações na topologia da rede, assegurando um encaminhamento fiável e eficiente, sendo a sua configuração essencial para o correto funcionamento da comunicação entre diferentes redes.

Configurações OSPF (Routers de distribuição):

Sede - Águeda
router ospf 1 router-id 1.1.1.1 passive-interface default network 10.1.10.0 0.0.0.15 area 0 network 10.1.20.0 0.0.0.15 area 0 network 10.1.15.0 0.0.0.15 area 0 network 10.1.30.0 0.0.0.15 area 0 network 10.1.100.0 0.0.0.255 area 0 network 10.1.120.0 0.0.0.63 area 0 network 10.1.130.0 0.0.0.63 area 0

Aveiro – Polo 1
router ospf 1 router-id 2.2.2.2 passive-interface default network 10.2.10.0 0.0.0.15 area 0 network 10.2.11.0 0.0.0.15 area 0 network 10.2.12.0 0.0.0.31 area 0 network 10.2.13.0 0.0.0.63 area 0 network 10.2.14.0 0.0.0.63 area 0 network 10.2.15.0 0.0.0.63 area 0 network 10.2.20.0 0.0.1.255 area 0

OAZ – Polo 2
router ospf 1 router-id 3.3.3.3 passive-interface default network 10.3.20.0 0.0.0.31 area 0 network 10.3.25.0 0.0.0.15 area 0 network 10.3.100.0 0.0.0.255 area 0 network 10.3.120.0 0.0.0.63 area 0 network 10.3.130.0 0.0.0.63 area 0

NAT e Access-Lists:

O NAT (Network Address Translation) é um mecanismo de rede que permite que dispositivos com endereços IP privados comuniquem com a Internet através de um único endereço IP público. É configurado num router e funciona substituindo o endereço IP privado pelo endereço IP público quando o tráfego sai da rede interna.

Neste processo, são frequentemente utilizadas access lists (ACLs), que servem para definir quais os endereços IP ou redes internas que estão autorizados a utilizar o NAT. As access lists permitem controlar e filtrar o tráfego, especificando que dispositivos podem ter o seu endereço traduzido, garantindo assim um maior controlo e organização da comunicação.

Configuração do NAT e suas Access-Lists:

Sede - Águeda
interface FastEthernet0/0 ip address 10.1.250.1 255.255.255.252 ip nat inside ip virtual-reassembly duplex auto speed auto interface FastEthernet0/1 ip address dhcp ip nat outside ip virtual-reassembly duplex auto speed auto ip nat inside source static udp 10.1.120.10 1194 interface FastEthernet0/1 1194 ip nat inside source static tcp 192.168.229.191 443 interface FastEthernet0/1 443 ip nat inside source static tcp 192.168.229.191 80 interface FastEthernet0/1 80 ip nat inside source route-map NAT interface FastEthernet0/1 overload ip access-list extended NO_NAT deny ip 10.1.0.0 0.0.255.255 10.2.0.0 0.0.255.255 deny ip 10.1.0.0 0.0.255.255 10.3.0.0 0.0.255.255 permit ip 10.1.0.0 0.0.255.255 any access-list 1 permit 10.3.0.0 0.0.255.255 access-list 1 permit 10.2.0.0 0.0.255.255 access-list 1 permit 10.1.0.0 0.0.255.255 access-list 1 permit 192.0.0.0 0.255.255.255 access-list 100 permit ip 10.1.0.0 0.0.0.255 10.3.0.0 0.0.0.255 access-list 101 permit ip 10.1.0.0 0.0.0.255 10.2.0.0 0.0.0.255

Aveiro – Polo 1

```
interface FastEthernet0/0
 ip address dhcp
 ip nat outside
 ip virtual-reassembly
 duplex auto
 speed auto

interface FastEthernet0/1
 ip address 10.2.251.2 255.255.255.252
 ip nat inside
 ip virtual-reassembly
 speed 100
 full-duplex

interface FastEthernet2/0
 ip address 10.2.254.1 255.255.255.252
 ip nat inside
 ip virtual-reassembly
 duplex auto
 speed auto
ip access-list extended NO_NAT
 deny   ip 10.1.0.0 0.0.255.255 10.2.0.0 0.0.255.255
 deny   ip 10.1.0.0 0.0.255.255 10.2.254.0 0.0.0.3
 permit ip 10.1.0.0 0.0.255.255 any

access-list 1 permit 10.3.0.0 0.0.255.255
access-list 1 permit 10.1.0.0 0.0.255.255
access-list 1 permit 10.2.0.0 0.0.255.255
access-list 1 permit 192.0.0.0 0.255.255.255
access-list 1 permit 10.4.0.0 0.0.0.3
access-list 100 permit ip 10.2.0.0 0.0.0.255 10.1.0.0 0.0.0.255
```

OAZ – Polo 2

```
interface FastEthernet0/0
 ip address dhcp
 ip nat outside
 ip virtual-reassembly
 duplex auto
 speed auto

interface FastEthernet0/1
 ip address 10.3.251.2 255.255.255.252
 ip nat inside
 ip virtual-reassembly
 duplex auto
 speed auto

interface FastEthernet2/0
 ip address 10.3.252.1 255.255.255.252
 ip nat inside
 ip virtual-reassembly
 duplex auto
 speed auto

ip access-list standard NAT-INSIDE
ip access-list standard NAT_INSIDE
 permit 10.3.0.0 0.0.255.255

access-list 1 permit 10.3.0.0 0.0.255.255
access-list 1 permit 10.2.0.0 0.0.255.255
access-list 1 permit 10.1.0.0 0.0.255.255
access-list 1 permit 192.0.0.0 0.255.255.255
access-list 100 permit ip 10.3.0.0 0.0.0.255 10.1.0.0 0.0.0.255
```

Configurações DHCP e DNS

DHCP

Operar com um servidor DHCP no Linux, como o isc-dhcp-server, representa um método eficiente de assegurar que os dispositivos obtenham automaticamente os endereços IP e outros parâmetros de rede.

A seguir, algumas razões para considerar:

- Ambiente flexível e modular - o Linux é muito adequado para funções de infraestrutura
- Estabilidade e desempenho elevados
- Integração fácil com VLANs e encaminhamento
- Baixo uso de recursos
- Custo reduzido, já que é open source e não tem necessidade de licenças

As funções do DHCP são:

- Atribuição automática de IP, máscara, gateway e DNS
- Reserva para dispositivos específicos
- Configuração via VLANs, o que possibilita sub-redes diferentes

Usar o Linux para DHCP não apenas complementa a infraestrutura Windows, mas ajuda a reduzir os custos e faz com que a entrega de IPs seja mais confiável.

Configuração DHCP

Instalação:

- sudo apt update
- sudo apt install isc-dhcp-server
- sudo systemctl start isc-dhcp-server
- sudo systemctl status isc-dhcp-server

Configuração do ficheiro dhcpd.conf:

DHCPD.CONF

```
# --- Águeda - Sede ---
option domain-name-servers 192.168.229.133;

# VLAN 10 - Administração
subnet 10.1.10.0 netmask 255.255.255.240 {
    option routers 10.1.10.2;
    option subnet-mask 255.255.255.240;
    option domain-name "agueda.office.moliceiro3.info";
    range 10.1.10.3 10.1.10.14;
}

# VLAN 20 - Desenvolvimento
subnet 10.1.20.0 netmask 255.255.255.240 {
    option routers 10.1.20.2;
    option subnet-mask 255.255.255.240;
    option domain-name "agueda.office.moliceiro3.info";
    range 10.1.20.3 10.1.20.14;
}

# VLAN 15 - Publicidade
subnet 10.1.15.0 netmask 255.255.255.240 {
    option routers 10.1.15.2;
    option subnet-mask 255.255.255.240;
    option domain-name "agueda.office.moliceiro3.info";
    range 10.1.15.3 10.1.15.14;
}

# VLAN 30 - Guias_Animadores
subnet 10.1.30.0 netmask 255.255.255.240 {
    option routers 10.1.30.1;
    option subnet-mask 255.255.255.240;
   option domain-name "agueda.office.moliceiro3.info";
    range 10.1.30.2 10.1.30.14;
}
# VLAN 100 - Guests
subnet 10.1.100.0 netmask 255.255.255.0 {
    option routers 10.1.100.1;
```

```
    option subnet-mask 255.255.255.0;
    option domain-name "agueda.office.moliceiro3.info";
    range 10.1.100.2 10.1.100.254;
}

# VLAN 200 - Infra_Servidores
subnet 10.1.120.0 netmask 255.255.255.192 {
    option routers 10.1.120.1;
    option subnet-mask 255.255.255.192;
    option domain-name "agueda.office.moliceiro3.info";
    range 10.1.120.3 10.1.120.62;
}

#------ Aveiro - Polo 1 -------

# VLAN 10 - Administração (10.2.10.0/28)
subnet 10.2.10.0 netmask 255.255.255.240 {
    option routers 10.2.10.1;
    option subnet-mask 255.255.255.240;
    option domain-name "aveiro.office.moliceiro3.info";
    range 10.2.10.3 10.2.10.14;
}

# VLAN 30 - Guias_Animadores (10.2.12.0/27)
subnet 10.2.30.0 netmask 255.255.255.240 {
    option routers 10.2.30.1;
    option subnet-mask 255.255.255.224;
    option domain-name "aveiro.office.moliceiro3.info";
    range 10.2.30.3 10.2.30.14;

# VLAN 5 - Financeiro (10.2.11.0/28)
subnet 10.2.5.0 netmask 255.255.255.240 {
    option routers 10.2.5.1;
    option subnet-mask 255.255.255.240;
    option domain-name "aveiro.office.moliceiro3.info";
    range 10.2.5.3 10.2.5.14;
}
# VLAN 35 - VoIP (10.2.13.0/26)
subnet 10.2.35.0 netmask 255.255.255.240 {
    option routers 10.2.13.1;
```

```
    option subnet-mask 255.255.255.240;
    option domain-name "aveiro.office.moliceiro3.info";
    range 10.2.35.3 10.2.35.14;
}

# VLAN 100 - Guests (10.2.20.0/23)
subnet 10.2.100.0 netmask 255.255.255.0 {
    option routers 10.2.100.1;
    option subnet-mask 255.255.255.0;
    option domain-name "aveiro.office.moliceiro3.info";
    range 10.2.100.2 10.2.100.254;
}

#VLAN 300 - Wi-Fi  (10.2.15.0/26)
subnet 10.2.130.0 netmask 255.255.255.192 {
    option routers 10.2.130.1;
    option subnet-mask 255.255.255.192;
    option domain-name "aveiro.office.moliceiro3.info";
    range 10.2.130.3 10.2.130.62;
}

#------ OAZ - Polo 2 ------

VLAN 20 - Desenvolvimento (10.3.20.0/27)
subnet 10.3.20.0 netmask 255.255.255.224 {
    option routers 10.3.20.1;
    option subnet-mask 255.255.255.224;
    option domain-name "azemeis.office.moliceiro3.info";
    range 10.3.20.3 10.3.20.30;
}

# VLAN 25 - Recursos_Humanos (10.3.25.0/28)
subnet 10.3.25.0 netmask 255.255.255.240 {
    option routers 10.3.25.1;
    option subnet-mask 255.255.255.240;
    option domain-name "azemeis.office.moliceiro3.info";
    range 10.3.25.4 10.3.25.14;
}

# VLAN 100 - Guests (10.3.100.0/28)
```

```
subnet 10.3.100.0 netmask 255.255.255.240 {
    option routers 10.3.100.3;
    option subnet-mask 255.255.255.240;
    option domain-name "azemeis.office.moliceiro3.info";
    range 10.3.100.4 10.3.100.14;
}

#VLAN 300 - Wi-fi (10.3.130.0/26)
subnet 10.3.130.0 netmask 255.255.255.192 {
    option routers 10.3.130.3;
    option subnet-mask 255.255.255.192;
    option domain-name "azemeis.office.moliceiro3.info";
    range 10.3.130.4 10.3.130.62;
}
```

DNS Interno no Windows Server

Utilizar um servidor DNS no Windows Server é uma forma eficaz de gerir, de maneira centralizada, os nomes e recursos de uma rede empresarial.

O Windows Server possui o serviço DNS integrado ao Active Directory, o que facilita a resolução automática de nomes de máquinas, serviços e políticas dentro do domínio.

Essa integração permite que os computadores membros do domínio registem automaticamente os seus nomes e endereços IP no DNS, para reduzir a necessidade de configuração manual e minimizando erros administrativos.

As principais vantagens desta configuração incluem:

- Integração com o Active Directory, permitindo o registo automático dos hosts do domínio
- Resolução eficiente de nomes internos da rede
- Gestão centralizada
- Registos dinâmicos, onde as máquinas atualizam automaticamente os seus registos DNS quando o endereço IP é alterado

Essas funcionalidades aumentam a confiabilidade da rede, simplificam a administração e melhoram tanto a segurança quanto a consistência dos registos DNS internos.

Dns Externo

No Windows Server, o DNS Reverso é responsável pela resolução inversa de nomes, ou seja, pela tradução de um endereço IP para um nome de host. Enquanto o DNS direto converte nomes de domínio em endereços IP, o DNS reverso realiza exatamente o processo oposto.

Essa funcionalidade é implementada através das chamadas Reverse Lookup Zones, que mapeiam endereços IP públicos para nomes de domínio (Moliceiro3.info) através de registos PTR.

O DNS reverso externo é fundamental para garantir a reputação do domínio na internet.

Implementação da VPN Site-to-Site (IPsec)

A interligação das redes entre os três polos (Águeda, Oliveira de Azeméis e Aveiro) foi estabelecida através de túneis IPsec nos routers Cisco, garantindo uma conexão segura e lógica entre redes geograficamente dispersas.

O IPsec foi selecionado pela sua capacidade robusta de encapsular e encriptar o tráfego de dados sobre a rede pública, permitindo que as redes privadas operem como se estivessem sob a mesma infraestrutura controlada.

A construção dos túneis envolveu duas fases de configuração:

- Fase 1 ou ISAKMP, estabeleceu-se um canal de gestão seguro e autenticado entre os gateways. Este processo foi suportado pela definição de uma política de segurança específica, a "policy 10", que determinou a utilização do algoritmo AES para a cifragem dos dados de controlo.
- Fase 2, focada na definição dos parâmetros de proteção para o tráfego de dados efetivo, foram criados conjuntos de transformação (transform-sets) que especificam a combinação do protocolo ESP com cifragem AES e o algoritmo SHA-HMAC para assegurar a integridade e a autenticação de cada pacote. Optou-se pela operação em modo túnel, o que permite o encapsulamento completo do pacote IP original, protegendo não apenas os dados, mas também o endereçamento interno das redes.

É fundamental notar a utilização do protocolo OSPF na interface do túnel (ip ospf 3 area 0). Esta justificação técnica permite que as redes locais dos polos (10.x.x.x) sejam trocadas dinamicamente sobre o canal seguro do IPsec. O OSPF garante a rapidez de convergência e a otimização do encaminhamento entre as localizações.

Configurações dos Tuneis IPSEC:

Águeda
crypto isakmp policy 10
encr aes
authentication pre-share
group 2
crypto isakmp key grupo3.agueda address 192.168.229.19
crypto isakmp key ag_av address 192.168.229.17
crypto isakmp key ag_oaz2 address 192.168.229.33
crypto ipsec transform-set TUNNEL-SET esp-aes esp-sha-hmac
crypto ipsec transform-set grupo3.agueda esp-aes esp-sha-hmac
crypto ipsec transform-set ag_av esp-aes esp-sha-hmac
crypto ipsec transform-set ag_oaz esp-aes esp-sha-hmac
crypto ipsec transform-set ag_oaz2 esp-aes esp-sha-hmac
crypto ipsec profile ag-aoz
set transform-set grupo3.agueda
crypto ipsec profile ag-av
set transform-set ag_av
crypto ipsec profile ag-oaz
set transform-set ag_oaz
crypto ipsec profile ag-oaz1
set transform-set grupo3.agueda
crypto ipsec profile ag-oaz2
set transform-set ag_oaz2
crypto map VPN-MAP 10 ipsec-isakmp
set peer 192.168.229.187
set transform-set TUNNEL-SET
match address 100
interface Tunnel1

```
ip address 10.4.140.1 255.255.255.252
ip ospf mtu-ignore
ip ospf 1 area 0
tunnel source FastEthernet0/1
tunnel destination 192.168.229.19
tunnel mode ipsec ipv4
tunnel protection ipsec profile ag-oaz

interface Tunnel2
ip address 10.4.141.1 255.255.255.252
ip ospf mtu-ignore
ip ospf 1 area 0
tunnel source FastEthernet0/1
tunnel destination 192.168.229.17
tunnel mode ipsec ipv4
tunnel protection ipsec profile ag-av

interface Tunnel3
ip address 10.1.143.1 255.255.255.252
ip ospf mtu-ignore
ip ospf 1 area 0
tunnel source FastEthernet0/1
tunnel destination 192.168.229.33
tunnel mode ipsec ipv4
tunnel protection ipsec profile ag-oaz2

interface Tunnel230
no ip address
tunnel source FastEthernet0/1
tunnel destination 192.168.229.191
```

Aveiro

```
crypto isakmp policy 10
 encr aes
 authentication pre-share
 group 2
crypto isakmp key ag_av address 192.168.229.146
crypto isakmp key av_oaz address 192.168.229.19

crypto ipsec transform-set TS esp-aes esp-sha-hmac
crypto ipsec transform-set ag_av esp-aes esp-sha-hmac
crypto ipsec transform-set av_oaz esp-aes esp-sha-hmac

crypto ipsec profile ag-av
 set transform-set ag_av

crypto ipsec profile av-oaz
 set transform-set av_oaz

interface Tunnel1
 ip address 10.4.141.2 255.255.255.252
 ip ospf mtu-ignore
 ip ospf 1 area 0
 tunnel source FastEthernet0/0
 tunnel destination 192.168.229.146
 tunnel mode ipsec ipv4
 tunnel protection ipsec profile ag-av

interface Tunnel2
 ip address 10.4.142.1 255.255.255.252
 ip ospf mtu-ignore
 ip ospf 1 area 0
 tunnel source FastEthernet0/0
 tunnel destination 192.168.229.19
 tunnel protection ipsec profile av-oaz
```

OAZ

```
crypto isakmp policy 10
 encr aes
 authentication pre-share
 group 2
crypto isakmp key grupo3.agueda address 192.168.229.146
crypto isakmp key av_oaz address 192.168.229.17

crypto ipsec transform-set TS esp-aes esp-sha-hmac
crypto ipsec transform-set grupo3.agueda esp-aes esp-sha-hmac
crypto ipsec transform-set av_oaz esp-aes esp-sha-hmac

crypto ipsec profile av-oaz
 set transform-set av_oaz

crypto ipsec profile oaz-ag
 set transform-set grupo3.agueda

interface Tunnel1
 ip address 10.4.140.2 255.255.255.252
 ip ospf mtu-ignore
 ip ospf 1 area 0
 tunnel source FastEthernet0/0
 tunnel destination 192.168.229.146
 tunnel mode ipsec ipv4
 tunnel protection ipsec profile oaz-ag

interface Tunnel2
 ip address 10.4.142.2 255.255.255.252
 ip ospf mtu-ignore
 ip ospf 1 area 0
 tunnel source FastEthernet0/0
 tunnel destination 192.168.229.17
 tunnel mode ipsec ipv4
 tunnel protection ipsec profile av-oaz
```

Access Point (AP):

Um AP é um dispositivo de rede que permite a ligação de equipamentos sem fios, como computadores portáteis e dispositivos móveis, a uma rede local. O seu principal objetivo é disponibilizar acesso à rede através de tecnologia Wi-Fi, eliminando a necessidade de ligações por cabo.

Configurações do AP:

Access Point
aaa group server radius rad_eap server 10.1.120.10 auth-port 1812 acct-port 1813 aaa group server radius rad_mac aaa group server radius rad_acct server 10.1.120.10 auth-port 1812 acct-port 1813 aaa group server radius rad_admin aaa group server tacacs+ tac_admin aaa group server radius rad_pmip aaa group server radius dummy aaa authentication login default local aaa authentication login eap_methods group radius aaa authentication login mac_methods local aaa authentication dot1x default group rad_eap aaa authorization exec default local aaa authorization network default group radius aaa accounting network default start-stop group radius aaa accounting network acct_methods start-stop group rad_acct aaa session-id common

```
dot11 ssid Moliceiros3
   vlan 300
   authentication open eap eap_methods
   authentication key-management wpa
   mbssid guest-mode

dot11 ssid Moliceiros3_Guests
   vlan 100
   authentication open
   authentication key-management wpa
   mbssid guest-mode
   wpa-psk ascii 7 105F1E1C17030B5A5E577E

dot11 ssid SSID_TESTE
   vlan 200
   authentication open eap eap_methods
   authentication key-management wpa
       accounting optional
   guest-mode

interface FastEthernet0.1
 encapsulation dot1Q 300 native
 no ip route-cache
 bridge-group 1
 no bridge-group 1 source-learning
 bridge-group 1 spanning-disabled

interface FastEthernet0.2
 encapsulation dot1Q 100
 no ip route-cache
 bridge-group 2
 no bridge-group 2 source-learning
 bridge-group 2 spanning-disabled

interface FastEthernet0.3
 encapsulation dot1Q 200
 no ip route-cache
 bridge-group 3
 no bridge-group 3 source-learning
```

```
 bridge-group 3 spanning-disabled


interface BVI1
 ip address 10.1.130.8 255.255.255.0
 no ip route-cache
```

Configuração e Implementação VPN:

A solução de conectividade segura implementada visa garantir a interligação contínua e a proteção dos dados entre os sites da Moliceiros da Ria, bem como fornecer acesso seguro aos colaboradores em regime remoto. A implementação integra duas arquiteturas distintas: uma VPN site-to-site (IPsec) e uma VPN de acesso remoto (OpenVPN). Toda a infraestrutura utiliza routers Cisco, servidor Windows e um servidor OpenVPN baseado em Linux.

Configuração da VPN de Acesso Remoto (OpenVPN)

A VPN de acesso remoto foi configurada no Linux utilizando o serviço OpenVPN, instalado através de easy-openvpn-server (via snap), providenciando uma solução flexível e de código aberto para utilizadores externos.

Configurações Chave:

Protocolos: UDP 1194 (principal) e TCP 443 (opcional, para contornar restrições de firewall em redes externas).

Segurança: Geração e exportação de certificados individuais para cada utilizador.

Controlo de Tráfego: Utilização do comando push-default-gateway para forçar todo o tráfego dos clientes remotos a passar pelo túnel, garantindo que o acesso à Internet e aos recursos internos seja feito sob a política de segurança da empresa.

Comandos Utilizados:

- sudo snap install easy-openvpn-server
- sudo easy-openvpn-server.init
- sudo openvpn --config cliente.ovpn

Testes de Validação:

- netstat -tulnp | grep openvpn e lsof -i :1194 / :443 (Confirmação de que o serviço está a escutar)
- nc -u -vz IP 1194 e nc -vz IP 443 (Testes de acessibilidade a partir da rede externa)

NAT e Port-Forwarding no Router Cisco

Para permitir que o servidor **OpenVPN** (localizado na rede interna, na sub-rede 10.1.120.x) fosse acessível a partir da Internet, foi necessário configurar o Port-Forwarding no router de Bordo através de NAT Estático.

As regras de NAT Estático garantem a tradução das portas públicas para o IP privado do servidor:

Comando	Protocolo	IP Servidor Interno	Porta Externa
ip nat inside source static	UDP	10.1.120.10	1194
ip nat inside source static	TCP	10.1.120.10	443

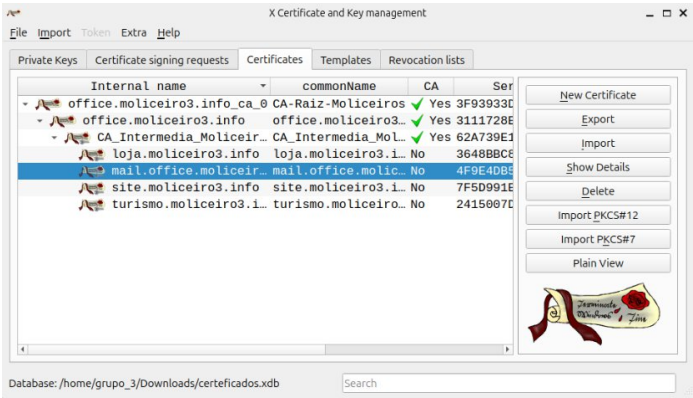
Validação:

- A acessibilidade pode ser validada com externos de netcat (nc) às portas UDP 1194 e TCP 443.
- O IP público de saída foi confirmado com curl ifconfig.me.
- As rotas e traduções foram verificadas com show ip route e show ip nat translations.

Servidor de Mail (IredMAIL):

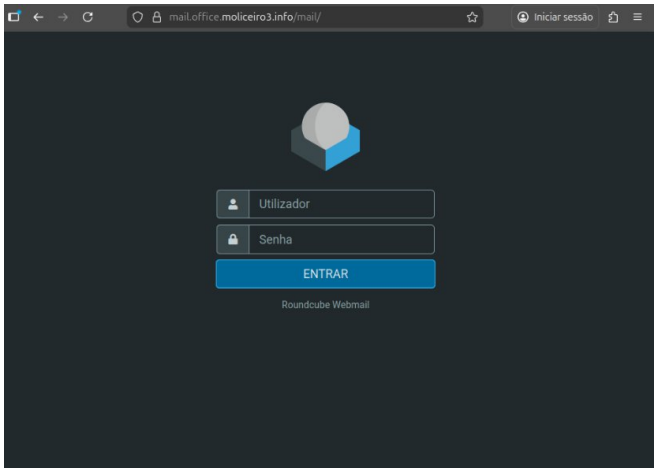
O iRedMail é uma solução completa de servidor Mail que permite implementar e gerir um sistema de email de forma centralizada e segura. Este software integra vários serviços essenciais, como o envio e receção de emails, a autenticação de utilizadores e a filtragem de spam, facilitando a administração de um servidor de correio numa rede.

Configuração do Servidor de Mail:

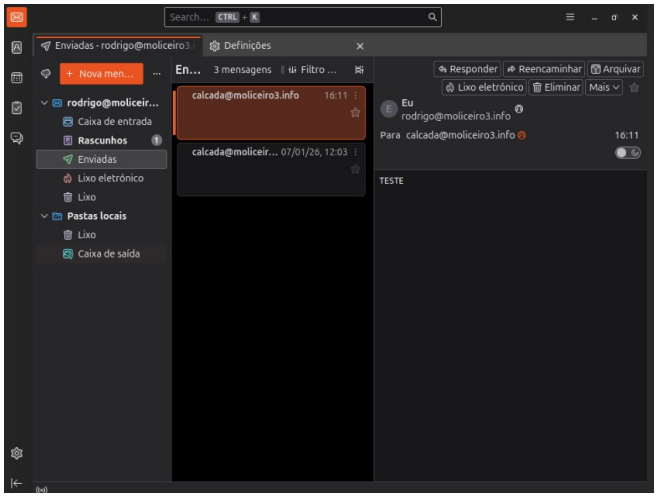
Servidor Mail
Instalação: sudo apt install -y wget curl gnupg2 wget https://github.com/iredmail/iRedMail/archive/refs/tags/1.4.0.tar.gz -O iRedMail.tar.gz tar xzf iRedMail.tar.gz IRedMail-* #Depois de entrar no GUI de instalção, configurámos: #Dominio de email: moliceiros3.info Depois de instalado: #Acedemos a página dele na web como administrador para criarmos emails para os membros do grupo https://mail.office.moliceiro3.info/iredmail https://mail.office.moliceiro3.info Certificados: #Depois de ter criado os certificados para o mail    #Vamos os importar para dentro da pasta /etc/nginx/templates/ssl.tmpl do Iredmail

```
GNU nano 7.2                    /etc/nginx/templates/ssl.tmpl
# Fix 'The Logjam Attack'.
ssl_ciphers EECDH+CHACHA20:EECDH+AESGCM:EDH+AESGCM:AES256+EECDH;
ssl_prefer_server_ciphers on;
ssl_dhparam /etc/ssl/dh2048_param.pem;

# Greatly improve the performance of keep-alive connections over SSL.
# With this enabled, client is not necessary to do a full SSL-handshake for
# every request, thus saving time and cpu-resources.
ssl_session_cache shared:SSL:10m;

# To use your own ssl cert (e.g. "Let's Encrypt"), please create symbol link to
# ssl cert/key used below, so that we can manage this config file with Ansible.
#
# For example:
#
# rm -f /etc/ssl/private/iRedMail.key
# rm -f /etc/ssl/certs/iRedMail.crt
# ln -s /etc/letsencrypt/live/<domain>/privkey.pem /etc/ssl/private/iRedMail.key
# ln -s /etc/letsencrypt/live/<domain>/fullchain.pem /etc/ssl/certs/iRedMail.crt
#
# To request free "Let's Encrypt" cert, please check our tutorial:
# https://docs.iredmail.org/letsencrypt.html
ssl_certificate /etc/ssl/certs/mail.office.moliceiro3.info.crt;
ssl_certificate_key /etc/ssl/private/mail.office.moliceiro3.info_key.pem;
```

#Isto serve para que o url do email fique em https e para não ficar http



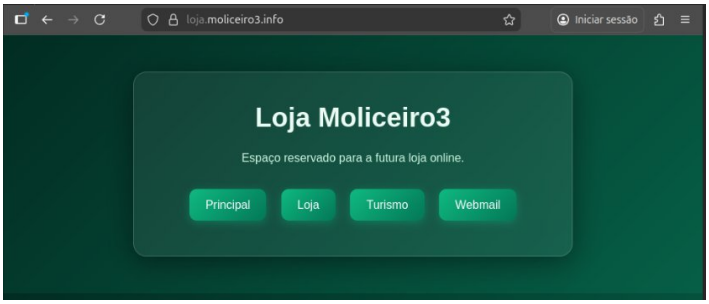
#Tabem temos de iniciar seção no Thunderbird



Servidor WEB:

Para servir de servidor Web usamos o ngnix, é um servidor web e proxy reverso utilizado para disponibilizar aplicações e websites de forma eficiente e segura.

Configurações Servidor Web:

Servidor Web
#Começamos por programar e colocar na web os nossos sites:  **#Depois fomos colocar os certificados para que os sites fiquem seguros:** Os certificados são importados para: /etc/ngnix/sites-enabled/" o url que o site tem"

Configuração VOIP – Asterisk:

Enquadramento:

No âmbito deste projeto foi implementado um sistema de comunicações **VoIP (Voice over IP)** baseado no protocolo **SIP**, utilizando o servidor **Kamailio** como proxy e registo SIP central. A solução foi integrada com uma base de dados **MariaDB** para autenticação dos utilizadores, com o serviço **DNS** do Windows Server para resolução de nomes e com a infraestrutura de rede previamente configurada da cidade de Águeda.

O objetivo principal consistiu em permitir o **registo de extensões SIP** e a **realização de chamadas VoIP**, tanto dentro da rede local como entre diferentes redes de outros grupos, recorrendo a autenticação baseada em base de dados e à descoberta do serviço SIP através de DNS.

Configuração do Kamailio:

O Kamailio foi configurado como servidor SIP responsável por aceitar pedidos de registo (REGISTER) e estabelecimento de chamadas (INVITE), assegurando a autenticação dos utilizadores e o encaminhamento correto das sessões SIP.

O serviço foi configurado para escutar em **UDP na porta 5060**, correspondente à porta SIP standard, aceitando ligações provenientes de qualquer interface de rede.

Ficheiro de configuração /etc/kamailio/kamailio.cfg:

Nas definições globais do Kamailio foram definidos os seguintes parâmetros:

- Ativação do modo de debug para facilitar a análise e diagnóstico de problemas
- Escuta em todas as interfaces de rede na porta SIP padrão (5060/UDP)

```
debug=3;
log_stderror=no;

listen=udp:0.0.0.0:5060;
```

Módulos Carregados:

Foram carregados apenas os módulos essenciais ao funcionamento do serviço SIP, garantindo simplicidade e eficiência da configuração:

- **tm.so** – Gestão de transações SIP
- **sl.so** – Envio de respostas SIP simples
- **rr.so** – Suporte a Record-Route
- **maxfwd.so** – Prevenção de loops SIP
- **textops.so** e **pv.so** – Manipulação de mensagens SIP
- **db_mysql.so** – Ligação à base de dados MySQL/MariaDB
- **usrloc.so** – Gestão da localização de utilizadores
- **registrar.so** – Gestão de registos SIP
- **auth.so** e **auth_db.so** – Autenticação SIP baseada em base de dados

Parâmetros dos Módulos:

O módulo **usrloc** foi configurado para armazenar os registos de utilizadores apenas em memória, simplificando a gestão e reduzindo a dependência de escrita em base de dados:

modparam("usrloc", "db_mode", 0)

A autenticação SIP foi configurada para utilizar a base de dados MariaDB, recorrendo à tabela subscriber:

```
modparam("auth_db", "db_url", "mysql://kamailio:kamailiorw@127.0.0.1:3306/kamailio")
modparam("auth_db", "calculate_ha1", yes)
modparam("auth_db", "password_column", "password")
modparam("auth_db", "use_domain", 1)
```

Lógica de Routing SIP:

A lógica de funcionamento do servidor SIP foi definida na secção request_route, responsável pelo tratamento de todos os pedidos recebidos.

Proteção contra Loops SIP:

Para evitar loops de encaminhamento SIP, foi configurada a verificação do cabeçalho **Max-Forwards**, rejeitando pedidos que excedam o número máximo permitido de saltos:

```
if (!mf_process_maxfwd_header("10")) {
    sl_send_reply("483", "Too Many Hops");
    exit;
}
```

Registo de Utilizadores (REGISTER):

O processo de registo SIP foi configurado da seguinte forma:

- Todos os pedidos REGISTER são autenticados contra a tabela subscriber
- Apenas utilizadores válidos conseguem registar-se
- Os registos são armazenados em memória

```
if (is_method("REGISTER")) {
    if (!auth_check("$fd", "subscriber", "1")) {
        auth_challenge("$fd", "0");
```

```
        exit;
    }
    save("location");
    sl_send_reply("200", "OK");
    exit;
}
```

Estabelecimento de Chamadas (INVITE):

Para o estabelecimento de chamadas SIP:

- O Kamailio procura o destino na tabela de localização
- Caso o utilizador esteja registado, a chamada é encaminhada
- Caso contrário, é devolvido um erro SIP 404

```
if (is_method("INVITE")) {
    if (!lookup("location")) {
        send_reply("404", "Not Found");
        exit;
    }
    t_relay();
    exit;
}
```

Base de Dados MariaDB:

Foi utilizada uma base de dados **MariaDB** denominada kamailio, responsável por armazenar as credenciais de autenticação SIP.

Na tabela subscriber foram criadas as seguintes extensões:

Username	Domain
101	grupo3.moliceiro3.info
102	grupo3.moliceiro3.info

Estas extensões foram utilizadas tanto por telefones SIP como por sistemas PBX externos, como o FreePBX.

Configuração de DNS:

O serviço DNS foi configurado para permitir a resolução completa do servidor SIP por nome.

Registos Diretos (A):

- grupo3.moliceiro3.info → 10.1.120.10

Registos Inversos (PTR):

- 10.1.120.10 → grupo3.moliceiro3.info

Registos SRV para SIP:

- _sip._udp.grupo3.moliceiro3.info → grupo3.moliceiro3.info:5060

Os testes efetuados com o comando dig confirmaram:

- Resolução direta (A)
- Resolução inversa (PTR)
- Descoberta do serviço SIP via SRV

Configuração de NAT no Router Cisco de Águeda

Para permitir comunicações VoIP a partir do exterior da rede, foi configurado **NAT** no router Cisco.

Encaminhamento da Porta SIP:

ip nat inside source static udp 10.1.120.10 5060 interface FastEthernet0/1 5060

Encaminhamento de RTP (Áudio):

Como o tráfego RTP utiliza um intervalo de portas, foi criada uma ACL específica:

```
ip access-list extended SIP-RTP
 permit udp any host 10.1.120.10 eq 5060
 permit udp any host 10.1.120.10 range 10000 20000
```

Aplicação do NAT com overload:

ip nat inside source list SIP-RTP interface FastEthernet0/1 overload

Esta configuração permitiu o correto encaminhamento do sinal SIP e do tráfego RTP, garantindo áudio bidirecional e mantendo o acesso à Internet.

Testes e Validação

Foram realizados com sucesso os seguintes testes:

- Registo de extensões SIP por IP e por domínio
- Estabelecimento de chamadas entre extensões
- Áudio bidirecional funcional (RTP)

- Resolução DNS direta, inversa e SRV
- Receção de chamadas de outras redes de grupos
- Envio de chamadas para redes externas de outros grupos

FreeRadius:

O FreeRADIUS é um servidor de autenticação e autorização (AAA) amplamente utilizado em redes, permitindo controlar o acesso de utilizadores e dispositivos a recursos de rede de forma centralizada e segura. A sua principal função é validar as credenciais dos utilizadores que tentam aceder à rede, assegurando que apenas quem possui autorização consegue estabelecer ligação.

Configuração FreeRadius:

Configurações no PC-Server

#Instalação:

sudo apt install freeradius freeradius-utils

#Configuração do ficheiro users do FreeRadius:

sudo nano /etc/freeradius/3.0/users

exemplos:

rodrigo Cleartext-Password := "Qwerty123@"

calcada Cleartext-Password := "Qwerty123@"

joao Cleartext-Password := "Qwerty123@"

#Configuração do ficheiro de clientes do FreeRadius

sudo nano /etc/freeradius/3.0/clients.conf

client ad_grupo3

Ipaddr = 10.1.130.8

secret = Qwerty123@

Shortname = ad_grupo3

Active Directory

Arquitetura Proposta e Tentativa de Implementação

A solução proposta baseia-se numa arquitetura cliente–autenticador–servidor, seguindo boas práticas utilizadas em redes empresariais. Esta abordagem permite separar claramente as funções de autenticação, controlo de acesso e infraestrutura de rede, facilitando a gestão e aumentando a segurança.

Arquitetura da Solução

A arquitetura prevista para a autenticação centralizada é composta pelos seguintes elementos:

- Active Directory (Samba AD): pensado para funcionar como repositório central de identidades, onde são armazenados os utilizadores, palavras-passe e informações do domínio.
- FreeRADIUS: configurado com o objetivo de atuar como servidor AAA, sendo responsável por receber os pedidos de autenticação 802.1X e validar as credenciais junto do Active Directory.
- Clientes finais: dispositivos dos utilizadores que tentam aceder à rede sem fios através de autenticação WPA-Enterprise.
- Esta separação de funções permite uma solução mais segura, escalável e organizada, alinhada com cenários reais de redes empresariais.

Implementação do Active Directory

Foi realizada uma instalação e configuração inicial do Active Directory baseado em Samba AD. O domínio foi criado e foram definidas contas de utilizador para efeitos de teste.

Foram também efetuadas algumas validações ao serviço de diretório, nomeadamente:

- Verificação da existência dos utilizadores no domínio
- Testes de autenticação utilizando a ferramenta ntlm_auth
- Confirmação da comunicação entre o servidor Linux e o domínio

Estes testes permitiram confirmar que o serviço de diretório funcionava de forma isolada. No entanto, o Active Directory não foi considerado operacional no estado final do projeto, uma vez que não foi possível validar a sua integração completa com os restantes serviços de autenticação.

Tentativa de Integração do FreeRADIUS com o Active Directory

O FreeRADIUS foi instalado e configurado com o objetivo de funcionar como servidor AAA centralizado, explorando a integração com o Active Directory através do protocolo MSCHAPv2 e do mecanismo ntlm_auth.

A ideia era permitir que o FreeRADIUS delegasse a validação das credenciais ao serviço de diretório, evitando a criação e gestão de utilizadores locais no próprio FreeRADIUS.

Apesar de terem sido realizados testes locais com ferramentas como radtest e ntlm_auth, a integração completa entre o FreeRADIUS e o Active Directory não ficou funcional no estado final do projeto, não sendo possível validar a autenticação 802.1X de ponta a ponta.

Por este motivo, esta funcionalidade é considerada trabalho exploratório, ficando identificada como uma possível melhoria futura.

Configuração do Access Point Cisco

O Access Point Cisco foi configurado para suportar autenticação AAA, utilizando o FreeRADIUS como servidor de autenticação remota.

Foram configurados dois SSIDs distintos:

- SSID Corporativo
- Segurança: WPA2/WPA3 Enterprise
- Autenticação: 802.1X
- Servidor RADIUS: FreeRADIUS
- VLAN dedicada associada ao SSID

A infraestrutura de rede encontra-se preparada para autenticação centralizada, ficando dependente da funcionalidade completa do serviço de autenticação, que não ficou operacional no estado final do projeto.

SSID de Convidados

- Segurança: WPA-PSK
- Autenticação: chave pré-partilhada
- Funcionamento independente de AAA

Este SSID encontra-se totalmente funcional, permitindo um acesso simples para utilizadores externos, sem depender de autenticação centralizada.

Backups Centralizados

Automação de Backups e Gestão de Redundância

Esta secção descreve a solução desenvolvida para a preservação sistemática das configurações dos Routers, Switches e Access Points. O sistema foi projetado para garantir a resiliência das configurações através da recolha automatizada de dados e do armazenamento georedundante.

Metodologia de Captura (Protocolo Telnet)

A extração das configurações é realizada através de um script Python que utiliza a biblioteca telnetlib. O fluxo de execução segue um protocolo rigoroso na interação com a linha de comando dos dispositivos:

Autenticação Automática: O script gere o login enviando credenciais de utilizador e palavra-passe conforme solicitado pelo terminal remoto.

Elevação de Privilégios: É executado o comando enable para aceder ao modo de configuração privilegiado, permitindo a leitura da configuração completa.

Normalização do Output: Utiliza-se o comando terminal length 0 para desativar a paginação, garantindo que a running-config seja capturada de forma contínua e sem interrupções de interface.

Lógica de Versionamento Local (Cleanup)

Para evitar o consumo excessivo de armazenamento e manter a organização dos ficheiros, o script implementa uma política de retenção:

Timestamping: Cada ficheiro de backup é nomeado com a data e hora exata da recolha (ex: R_NAT-AG-20260111-060000.cfg), permitindo auditorias temporais precisas.

Regra de Retenção (FIFO): O sistema mantém apenas as **3 versões mais recentes** de cada dispositivo. Sempre que um novo backup é criado, o script identifica e elimina automaticamente as versões mais antigas, garantindo que o histórico disponível seja sempre relevante.

Redundância em Nuvem (Integração Rclone)

A segurança dos dados é reforçada pela integração com a ferramenta **Rclone**, que providencia uma camada de backup *off-site*:

Sincronização Georedundante: Após a gravação local, o ficheiro é replicado para um repositório remoto no **Google Drive:** gdrive: Backups_Rede.

Disaster Recovery: Esta arquitetura assegura que as configurações críticas da rede possam ser recuperadas mesmo em caso de falha catastrófica no servidor de gestão local.

Monitorização e Diagnóstico

O script possui um sistema de logs em dois níveis para assegurar a fiabilidade da operação:

backup.log: Regista o sucesso das operações, uploads para a nuvem e remoção de ficheiros antigos.

debug.log: Captura detalhes técnicos e carimbos de início/fim de execução para facilitar o diagnóstico de eventuais falhas de conectividade ou timeouts.

Backup_redesv2.py

```
import telne lib
import time
import os
from datetime import datetime

LOG_FILE = "/home/grupo3/backup/backup.log"
BACKUP_DIR = "/home/grupo3/backups/"
MAX_VERSIONS = 3
RCLONE_REMOTE = "gdrive:Backups_Rede"
USERNAME = "grupo3"
PASSWORD = "grupo3"
ENABLE = "grupo3"

DEVICES = [
    {"host": "10.1.250.1", "name": "R_NAT-AG"},
    {"host": "10.1.250.2", "name": "R_DIST-AG"},
    {"host": "10.1.140.2", "name": "SW_AC-AG"},
    {"host": "10.1.130.8", "name": "AP-AG"},
    {"host": "10.2.251.2", "name": "R1_NAT-AV"},
    {"host": "10.2.251.1", "name": "R2_NAT-AV"},
    {"host": "10.2.253.2", "name": "R1_DIST-AV"},
    {"host": "10.2.5.1", "name": "R2_DIST-AV"},
    {"host": "10.3.251.2", "name": "R1_NAT-OAZ"},
    {"host": "10.3.251.1", "name": "R2_NAT-OAZ"},
    {"host": "10.3.252.2", "name": "R1_DIST-OAZ"},
    {"host": "10.3.253.2", "name": "R2_DIST-OAZ"}
]


def log(msg):
    with open(LOG_FILE, "a") as f:
        f.write(msg + "\n")

with open("/home/grupo3/backup/debug.log", "a") as f:
    f.write(f"SCRIPT INICIO {datetime.now()}\n")

os.makedirs(BACKUP_DIR, exist_ok=True)

def backup_telnet(device):
    try:
        tn = telnetlib.Telnet(device["host"], timeout=8)

        # Login
        tn.read_until(b"Username:", timeout=5)
        tn.write(USERNAME.encode() + b"\n")
```

```
        tn.read_until(b"Password:", timeout=5)
         tn.write(PASSWORD.encode() + b"\n")

        time.sleep(1)
        output = tn.read_very_eager().decode(errors="ignore")

        # Tentar enable (sem ficar preso)
        tn.write(b"enable\n")
          time.sleep(1)
        enable_output = tn.read_very_eager().decode(errors="ignore")

        if "Password" in enable_output:
            tn.write(ENABLE.encode() + b"\n")
            time.sleep(1)
            enable_output += tn.read_very_eager().decode(errors="ignore")

        # Verificar se entrou em modo privilegiado
        if "#" not in enable_output:
            log(f"[AVISO] {device['name']} sem enable — a continuar sem privilégio")

        # Evitar paginação
        tn.write(b"terminal length 0\n")
        time.sleep(1)

        tn.write(b"show running-config\n")
        time.sleep(4)

        config = tn.read_very_eager().decode(errors="ignore")
        tn.write(b"exit\n")
        tn.close()

        return config

    except Exception as e:
        log(f"[ERRO] Telnet em {device['name']} ({device['host']}): {e}")
        return None

def cleanup_old_backups(device_name):
    files = sorted(
        [f for f in os.listdir(BACKUP_DIR) if f.startswith(device_name)],
        reverse=True
    )
    for old in files[MAX_VERSIONS:]:
        os.remove(os.path.join(BACKUP_DIR, old))
        log(f"Backup antigo removido: {old}")

def save_and_upload(name, config):
    if not config or len(config) < 200:
        log("Config inválida, ignorado.")
        return

    timestamp = datetime.now().strftime("%Y%m%d-%H%M%S")
    filename = f"{name}-{timestamp}.cfg"
    filepath = os.path.join(BACKUP_DIR, filename)

    with open(filepath, "w") as f:
        f.write(config)

    log(f"Backup criado: {filepath}")
```

```
    cleanup_old_backups(name)

    os.system(f"rclone copy {filepath} {RCLONE_REMOTE}")
    log("Enviado para Google Drive")


for device in DEVICES:
    log(f"\n--- Backup {device['name']} ({device['host']}) ---")
    cfg = backup_telnet(device)
    if cfg:
        save_and_upload(device["name"], cfg)

log("\n--- PROCESSO TERMINADO ---")
```

Discord Bot-Ticketing

Esta parte do projeto foi desenhada para usar as dependencias de forma bidirecional, integrando a plataforma **Zabbix** com a rede social **Discord** através de uma aplicação personalizada em **Python**. A solução foi desenhada para garantir portabilidade e isolamento de dependências, utilizando a tecnologia de contentores **Docker** para a execução do bot. A integração baseia-se em dois fluxos principais de dados:

Fluxo de Entrada (Discord para Zabbix): Através de comandos de barra (/incident) no canal de texto, o utilizador envia dados diretamente para a API JSON-RPC do Zabbix. O script Python atua como um cliente de API, utilizando o método history.push para injetar os dados em itens do tipo **Trapper Zabbix**. A segurança desta comunicação é garantida via autenticação por *Bearer Tokens*.

Fluxo de Saída (Zabbix para Discord): Quando um Alerta/Trigger é ativado no Zabbix, o sistema utiliza um **Webhook** configurado para disparar um pedido HTTP POST para o contentor Docker. O bot, que corre um servidor **FastAPI/Uvicorn** na porta **8000**, processa o JSON recebido e formata uma notificação automática para o canal de texto do Discord, incluindo menções de severidade.

Infraestrutura em Docker

A escolha do **Docker Compose** permitiu a definição de uma infraestrutura como código (IaC), onde variáveis de ambiente críticas como tokens de API, URLs de servidores e IDs de canais são injetadas de forma segura no ambiente de execução. Esta abordagem facilita a escalabilidade do bot e assegura que o ambiente de desenvolvimento seja idêntico ao de produção, eliminando conflitos de bibliotecas locais.

Docker em Startup
sudo systemctl enable docker sudo systemctl start docker

Instalação Docker + Docker Compose
sudo apt update sudo apt install -y ca-certificates curl gnupg lsb-release sudo mkdir -p /etc/apt/keyrings

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | gpg --dearmor \
  -o /etc/apt/keyrings/docker.gpg

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" \
  | tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
```

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY bot ./bot

CMD ["python3", "bot/main.py"]
```

Ficheiro requirements.txt

```
discord.py
fastapi
uvicorn[standard]
asyncio
```

```
requests
```

Ficheiro docker-compose.yml

```
services:
  discord-bot:
    build: .
    container_name: discord-zabbix-bot
    restart: always
    environment:
      DISCORD_TOKEN:
MTQ0NTQzNjA0MTk4MDAyMjg3NA.GXPgMd.9utOmKxq2EymUtbXI9xotFNWvddo3huMSrQyAk
      CHANNEL_ID: 1445436478045032655
      ZABBIX_URL: http://10.1.120.10/zabbix/api_jsonrpc.php
      ZABBIX_API_TOKEN: 643145023ec9711739138383923646185 3ef02aa26031ed09fc00ddd2e8fc8c0
      ZABBIX_ITEM_ID: 71254
    ports:
      - "8000:8000"
    networks:
      - botnet

    networks:
      botnet:
        driver: bridge
```

Ps. Script Bot antes da Conclusão

NGINX

O servidor Nginx foi implementado como a camada de servidor web, sendo responsável por servir a interface de monitorização do Zabbix e atuar como o gateway de comunicação para as chamadas de API JSON-RPC provenientes do bot de Discord. Além de gerir o tráfego HTTP, o Nginx assegura que os pedidos de integração (webhooks) sejam corretamente encaminhados entre o backend do Zabbix e o serviço contentorizado em Docker.

Configurar NGINX como reverse proxy

```
sudo apt install nginx -y
```

Criação da configuração

server {
listen 8000;
server_name moliceiros3.info;
 	
location /zabbix {
proxy_pass http://127.0.0.1:8000/zabbix;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}

Ativação da configuração
ln -s /etc/nginx/sites-available/zabbix-bot.conf /etc/nginx/sites-enabled/ sudo nginx -t sudo systemctl restart nginx

Zabbix

Monitorização da Infraestrutura com Zabbix:

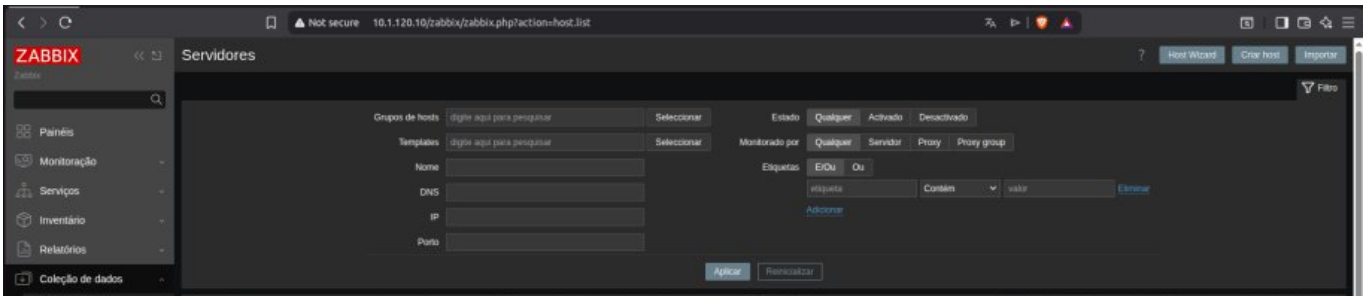
O Zabbix foi implementado como a plataforma central de monitorização da infraestrutura de rede da organização, permitindo acompanhar, em tempo real, o estado dos dispositivos e dos serviços distribuídos pelos vários segmentos e localizações.

Esta solução possibilita a deteção proativa de falhas, a análise do desempenho da rede e a obtenção de uma visão global do estado operacional dos equipamentos, contribuindo para uma gestão mais eficiente da infraestrutura.

Descoberta Automática de Dispositivos:

A monitorização da rede baseia-se no mecanismo de **descoberta automática (Network Discovery)** do Zabbix. Foram configuradas regras de descoberta automática para diferentes intervalos de endereços IP, correspondentes aos vários segmentos e localizações da infraestrutura.

Estas regras utilizam **ICMP Ping** para detetar dispositivos ativos na rede, permitindo que novos equipamentos sejam automaticamente identificados e adicionados ao sistema de monitorização, sem necessidade de configuração manual.

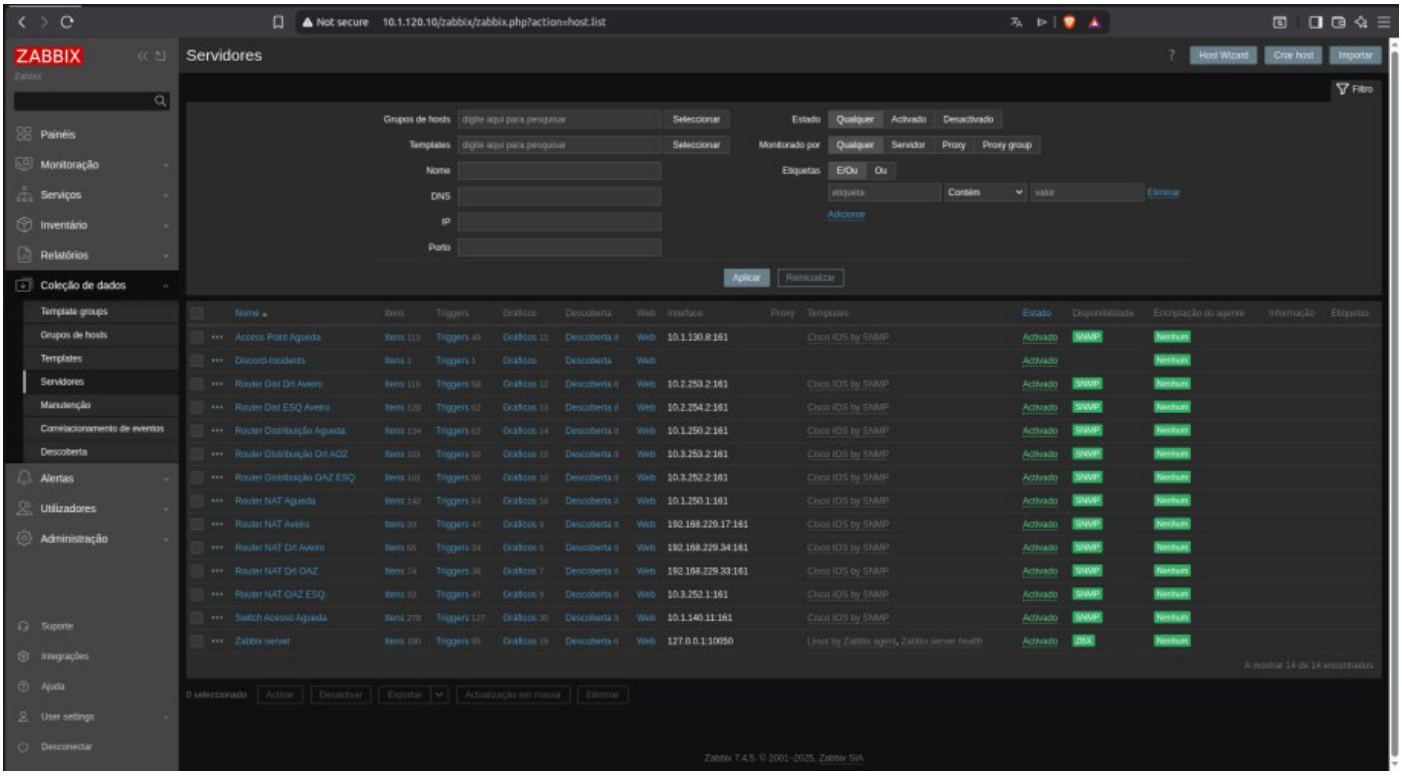


Regras de descoberta automática configuradas no Zabbix

Hosts Monitorizados:

Os dispositivos detetados através das regras de descoberta são automaticamente criados como **hosts** no Zabbix. Estes incluem routers, switches, pontos de acesso (*access points*) e servidores.

Cada host apresenta o respetivo estado de disponibilidade, permitindo identificar rapidamente equipamentos ativos, indisponíveis ou com problemas de comunicação.



Hosts monitorizados no Zabbix

Monitorização via SNMP:

A maioria dos equipamentos de rede é monitorizada através do protocolo **SNMP**, evitando a necessidade de instalação de agentes nos dispositivos.

São recolhidas métricas como:

- Tráfego das interfaces
- Erros de comunicação
- Pacotes descartados
- Estado das interfaces

Estas métricas são apresentadas sob a forma de gráficos, permitindo a análise do desempenho da rede ao longo do tempo.

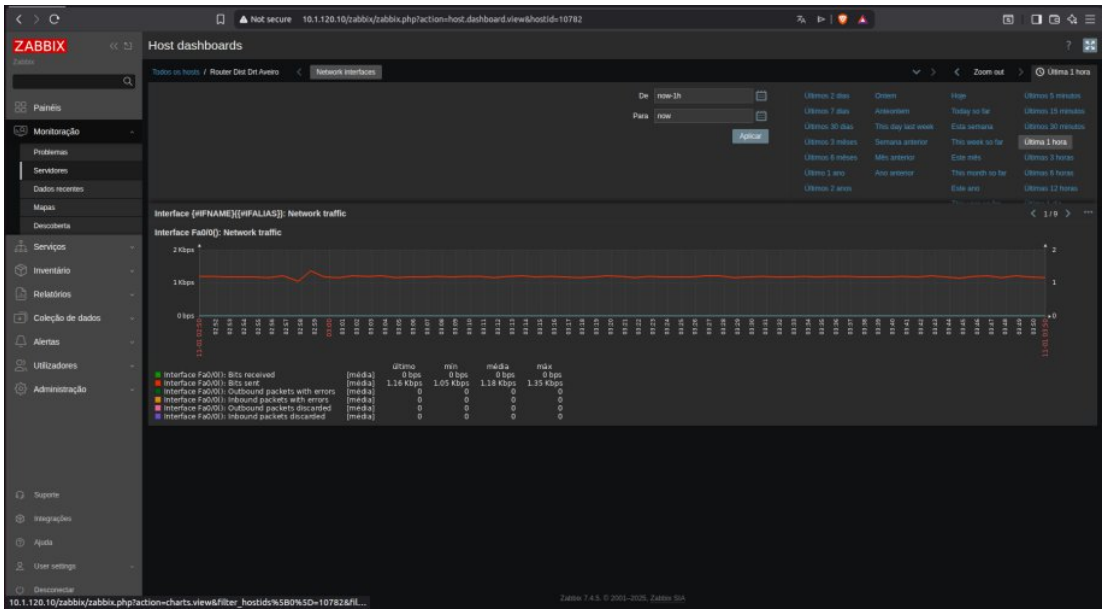
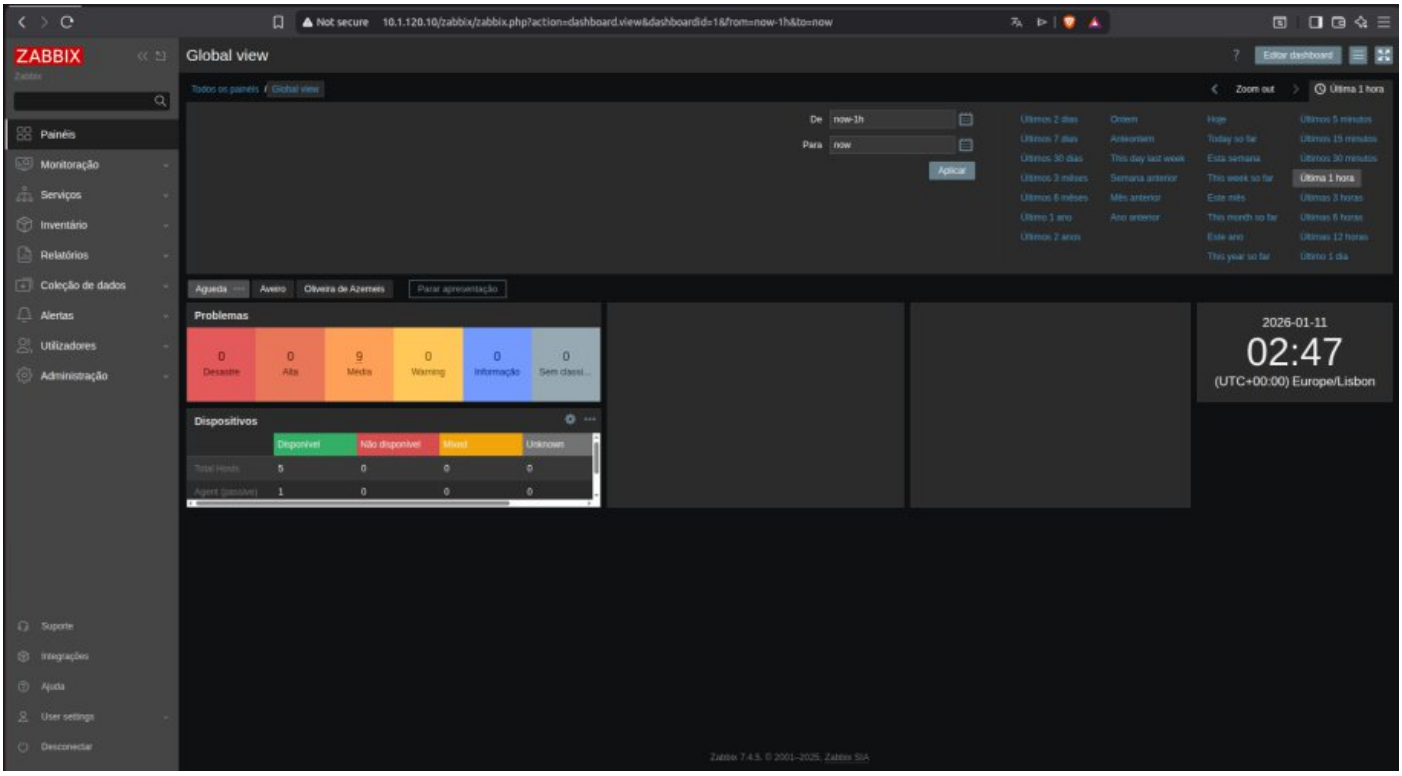


Gráfico de tráfego de uma interface monitorizada via SNMP

Painéis de Controlo e Visão Global:

Foram configurados **painéis de controlo (dashboards)** no Zabbix que permitem uma visão global do estado da infraestrutura. Estes painéis apresentam o número de problemas por nível de severidade, o estado dos dispositivos e a sua distribuição por localização.

Esta abordagem facilita a deteção rápida de falhas e a correta priorização de incidentes.



Painel de controlo global de monitorização no Zabbix

Integração Zabbix–Discord:

Com base nesta plataforma de monitorização, foi desenvolvida uma integração avançada de alertas com o Discord, permitindo o registo manual de incidentes e a notificação imediata das equipas responsáveis.

Esta integração foi implementada utilizando funcionalidades nativas do Zabbix, conforme descrito nas secções seguintes.

Modelo de Dados: Item do Tipo Trapper:

O item foi configurado com a chave discord-incidents e com o tipo de informação **Texto**. Este tipo de item permite que o Zabbix funcione de forma passiva, reduzindo o consumo de recursos de rede, ao processar dados apenas quando o bot desenvolvido em Python realiza o envio ativo da informação através da API.

Lógica de Disparo: Trigger e Estados de Incidente:

A **trigger (disparador)** é o componente responsável por transformar dados brutos em alertas acionáveis.

Neste caso, a trigger utiliza uma lógica de comparação de cadeias de caracteres, sendo ativada sempre que o item recebe qualquer conteúdo não vazio, ou seja, quando é enviada uma mensagem de texto com comprimento superior a zero.

Classificação de Severidade:

Foi atribuída à trigger a severidade **Alta (High)**. Esta escolha justifica-se pela necessidade de garantir que o mecanismo de ações do Zabbix prioriza este evento, permitindo o envio imediato de notificações.

A severidade elevada assegura que a notificação enviada para o Discord inclui a menção @everyone, garantindo visibilidade máxima do incidente.

Configuração do Host e da Trigger:

- **Host:** Discord-Incidents
- **Nome da trigger:** Incidente manual
- **Tipo de item:** Zabbix Trapper
- **Condição de ativação:** Receção de mensagem de texto com mais de zero caracteres

Integração de Saída: Webhook e Tipos de Media:

Neste ponto, o Zabbix atua como cliente, enviando notificações para sistemas externos.

Foi configurado um **Media Type (tipo de media)** do tipo Webhook, apontando para o endpoint da API exposta pelo contentor Docker:

- **Endpoint:** http://10.1.120.10:8000/zabbix

Para garantir o correto funcionamento da integração, foi necessário definir um **modelo de mensagem em formato JSON** no Zabbix. Este modelo constitui o *payload* da notificação enviada, sendo essencial para que o sistema externo consiga interpretar corretamente a informação.

Grafana

Integração do Grafana com o Zabbix e o Loki

Após a instalação e configuração inicial do Grafana, procedeu-se à integração desta plataforma com o Zabbix e, posteriormente, com o Loki, com o objetivo de centralizar a visualização de métricas e de registos (*logs*) num único sistema de monitorização.

Integração do Grafana com o Zabbix

Numa primeira fase, foi instalado no Grafana o plugin **Zabbix Data Source** (alexanderzobnin-zabbix-datasource), necessário para permitir a comunicação entre o Grafana e o Zabbix através da respetiva API. Este plugin possibilita a recolha das métricas monitorizadas pelo Zabbix e a sua representação gráfica através de painéis de controlo no Grafana.

De seguida, no Zabbix, foi criado um utilizador dedicado para o Grafana, associado a um grupo com permissões de leitura (*read-only*) sobre os hosts monitorizados. Esta abordagem aumenta o nível de segurança, evitando a utilização de contas administrativas em integrações externas.

No Grafana, foi então configurada uma nova fonte de dados do tipo Zabbix, onde foram definidos o endereço do servidor Zabbix e as credenciais do utilizador previamente criado, permitindo o acesso às métricas recolhidas.

Integração do Grafana com o Loki

Para além da monitorização de métricas, foi também implementada a recolha e visualização de registos (*logs*) através da integração do Grafana com o **Loki**. O Loki é uma solução de agregação de logs desenvolvida pela Grafana Labs, optimizada para funcionar de forma integrada com o Grafana.

Após a instalação do Loki, este foi configurado como uma nova fonte de dados no Grafana. Ao contrário de soluções tradicionais de gestão de logs, o Loki indexa apenas os metadados (*labels*) associados aos registos, tornando o sistema mais leve e eficiente em termos de armazenamento e desempenho.

Com o Loki em funcionamento, passou a ser possível:

- Centralizar os registos de serviços e sistemas
- Pesquisar logs através de filtros e *labels*
- Correlacionar logs com métricas e eventos apresentados nos painéis de controlo do Grafana

Esta integração permite uma análise mais completa do estado da infraestrutura, combinando métricas de desempenho recolhidas pelo Zabbix com registos detalhados de eventos e erros fornecidos pelo Loki.

Endpoint do Grafana:

Endpoint: http://10.1.120.10:3000/

Webhook Api

Tipo: Webhook

Host: {HOST.NAME}

Message: {ITEM.NAME}: {ITEM.VALUE}

Severity: {TRIGGER.SEVERITY}

Trigger: {TRIGGER.NAME}

Script:

```
try {
    var params = JSON.parse(value),
        req = new HttpRequest(),
        body = {};



    body.trigger = params.trigger;
    body.host = params.host;
    body.severity = params.severity;
    body.message = params.message;

    req.addHeader('Content-Type: application/json');



   var response = req.post('http://10.1.120.10:8000/zabbix', JSON.stringify(body));

    return 'OK';
} catch (error) {
    Zabbix.log(3, 'Erro no Webhook: ' + error);
    throw 'Falha: ' + error;
}
```

Bot Pyhton

Arquitetura Híbrida (Bot + Web Server):

Dois fluxos de dados distintos simultaneamente:

Cliente Discord (discord.py): Mantém uma ligação persistente via WebSockets com o servidor do Discord para ouvir comandos como o /incident.

Servidor Web (FastAPI/Uvicorn): Escuta na porta **8000** à espera de notificações (Webhooks) vindas do Zabbix.

O uso de asyncio (programação assíncrona) permite que o bot responda a utilizadores no Discord sem bloquear a receção de novos alertas vindos do Zabbix.

Fluxo de Execução do Comando /incident:

Parsing: O bot remove o prefixo /incident e extrai o texto da mensagem.

Encapsulamento JSON-RPC: O texto é inserido num objeto JSON seguindo a especificação da API do Zabbix , utilizando o método history.push.

Autenticação: O pedido é enviado com um Header de autorização Bearer Token, garantindo a integridade e segurança da transação.

Feedback: O bot confirma ao utilizador no Discord que o incidente foi criado, após receber o 200 OK da API.

Ambiente:

Variáveis de Ambiente: Não possui dados sensíveis (tokens/IPs) no código fonte. Estes são injetados pelo **Docker Compose**, o que facilita a portabilidade e segurança.
Logs e Monitorização: A biblioteca logging é utilizada para registar cada pedido recebido (POST /zabbix), o que é essencial para auditoria e resolução de problemas.

Resumo das Bibliotecas Utilizadas:

discord.py: Interação com a API do Discord.
FastAPI: Framework para criação do endpoint do Webhook.
Requests: Cliente HTTP para comunicar com a API do Zabbix.
Uvicorn: Servidor ASGI de alta performance para rodar a aplicação.

Código BOT (FastAPI + Discord.py)

./bot/main.py

```
import os
import discord
from discord.ui
import View, Button, Select
from fastapi import FastAPI, Request
import uvicorn
import asyncio
import logging
import requests
from typing import Optional
from datetime import datetime
import hashlib

DISCORD_TOKEN = os.getenv("DISCORD_TOKEN")
CHANNEL_ID = int(os.getenv("CHANNEL_ID"))
CHANNEL_ID_MANUAL = int(os.getenv("CHANNEL_ID_MANUAL"))

ZABBIX_URL = os.getenv("ZABBIX_URL")
ZABBIX_API_TOKEN = os.getenv("ZABBIX_API_TOKEN")
ZABBIX_ITEM_ID = str(os.getenv("ZABBIX_ITEM_ID"))

id_item = 1

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("discord-zabbix-bot")

intents = discord.Intents.default()
intents.message_content = True
client = discord.Client(intents=intents)
bot_ready = asyncio.Event()
```

Armazena informações de claims de alertas

```
claimed_alerts = {}
```

Dicionário para rastrear status dos incidentes

```
incident_statuses = {}
```

Estados disponíveis

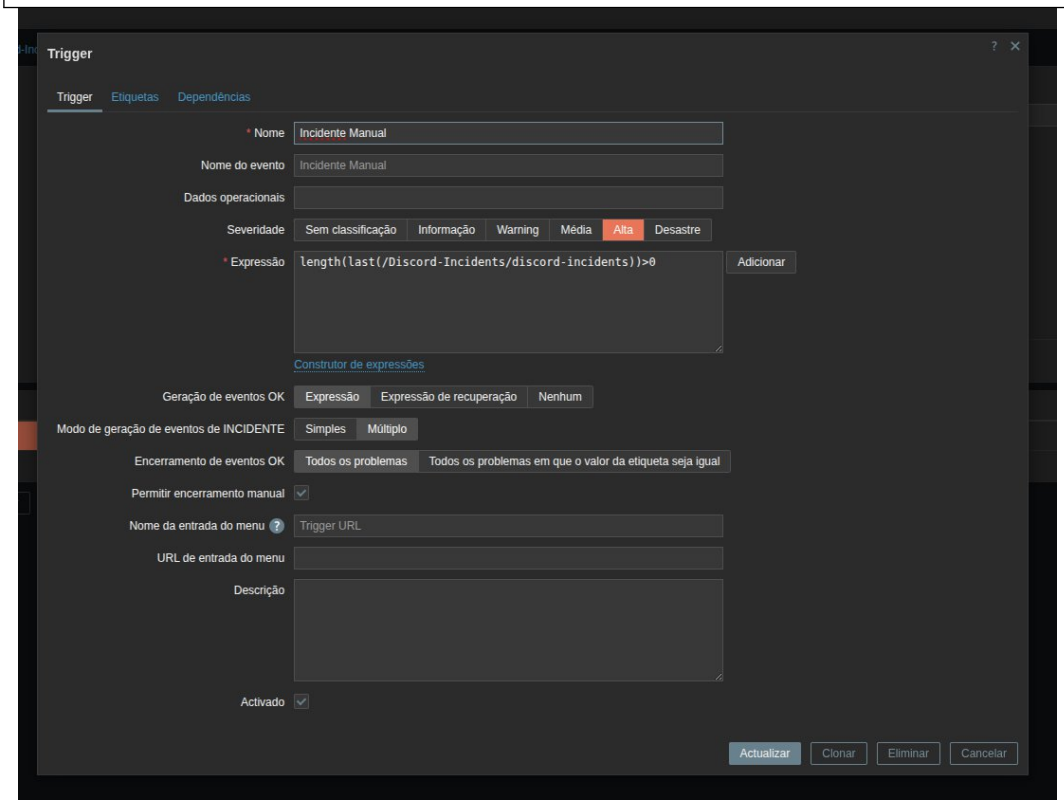
```
INCIDENT_STATUS = { "aberto": "🔴 Em Aberto", "investigacao": "🟡 Em Investigação", "resolvido": "✅ Resolvido" }

app = FastAPI()

class StatusSelect(Select):
def init(self, alert_id: str, alert_data: dict, claimed_by: Optional[discord.User] = None):
self.alert_id = alert_id
self.alert_data = alert_data
self.claimed_by = claimed_by

  options = [
```

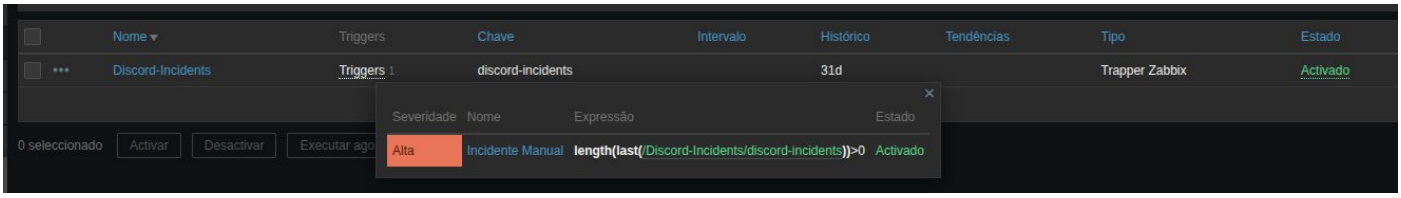
```
        discord.SelectOption(label=INCIDENT_STATUS["aberto"], value="aberto"),
        discord.SelectOption(label=INCIDENT_STATUS["investigacao"], value="investigacao"),
        discord.SelectOption(label=INCIDENT_STATUS["resolvido"], value="resolvido")
    ]

    super().__init__(
        placeholder="Alterar Status do Incidente...",
        min_values=1,
        max_values=1,
        options=options
    )

async def callback(self, interaction: discord.Interaction):
    new_status = self.values[0]

    # Armazena o novo status
    incident_statuses[self.alert_id] = {
        "status": new_status,
        "updated_by": interaction.user,
        "updated_at": datetime.now()
    }

    # Atualiza o embed da mensagem
    new_embed = create_alert_embed(self.alert_data, self.claimed_by)
    await interaction.response.defer()
    await interaction.message.edit(embed=new_embed)

    logger.info(f"Status do incidente {self.alert_id} alterado para {new_status} por {interaction.user}")


class ClaimView(View):
def init(self, alert_data: dict):
super().init(timeout=None)
self.alert_data = alert_data
self.claimed_by: Optional[discord.User] = None
self.alert_id = alert_data.get('id', 'unknown')

   # Adiciona o menu de status
    self.add_item(StatusSelect(self.alert_id, self.alert_data, self.claimed_by))

@discord.ui.button(label="Assumir", style=discord.ButtonStyle.green)
async def claim_button(self, interaction: discord.Interaction, button: Button):
    # Verifica se já foi assumido por alguém
    if self.claimed_by:
        await interaction.response.send_message(
            f"Este alerta já foi assumido por {self.claimed_by.mention}",
            ephemeral=True
        )
        return

    # Marca como assumido
    self.claimed_by = interaction.user

    # Desabilita o botão após o claim
    button.disabled = True

    # Atualiza a referência do StatusSelect
    for item in self.children:
        if isinstance(item, StatusSelect):
```

```
        item.claimed_by = self.claimed_by

    # Edita a mensagem original com a nova cor e informação
    embed = create_alert_embed(self.alert_data, interaction.user)
    await interaction.response.defer()
    await interaction.message.edit(embed=embed, view=self)

    logger.info(f"Alerta assumido por {interaction.user} ({interaction.user.id})")


def create_alert_embed(data: dict, claimed_by: Optional[discord.User] = None) -> discord.Embed:
"""Cria um embed formatado para o alerta do Zabbix""" severity = data.get('severity', '').lower() alert_id = data.get('id',
'unknown')

# Definir cor baseada na severidade
color_map = {
    'information': discord.Color.blue(),
    'warning': discord.Color.yellow(),
    'average': discord.Color.orange(),
    'high': discord.Color.red(),
    'disaster': discord.Color.dark_red(),
}
color = color_map.get(severity, discord.Color.greyple())

embed = discord.Embed(
    title="Alerta Zabbix",
    description=data.get('message', 'Sem descrição'),
    color=color
)

embed.add_field(name="Trigger", value=data.get('trigger', 'N/A'), inline=False)
embed.add_field(name="Host", value=data.get('host', 'N/A'), inline=True)
embed.add_field(name="Severidade", value=severity.upper(), inline=True)

if claimed_by:
    embed.add_field(
        name="Responsável",
        value=f" {claimed_by.mention}",
        inline=False
    )
else:
    embed.add_field(
        name="Responsável",
        value="Waiting...",
        inline=False
    )

# Mostra o status do incidente se existir
if alert_id in incident_statuses:
    status_info = incident_statuses[alert_id]
    status_label = INCIDENT_STATUS.get(status_info['status'], status_info['status'])
    embed.add_field(
        name="Status do Incidente",
        value=f"{status_label}",
        inline=False
    )
else:
    embed.add_field(
        name="Status do Incidente",
```

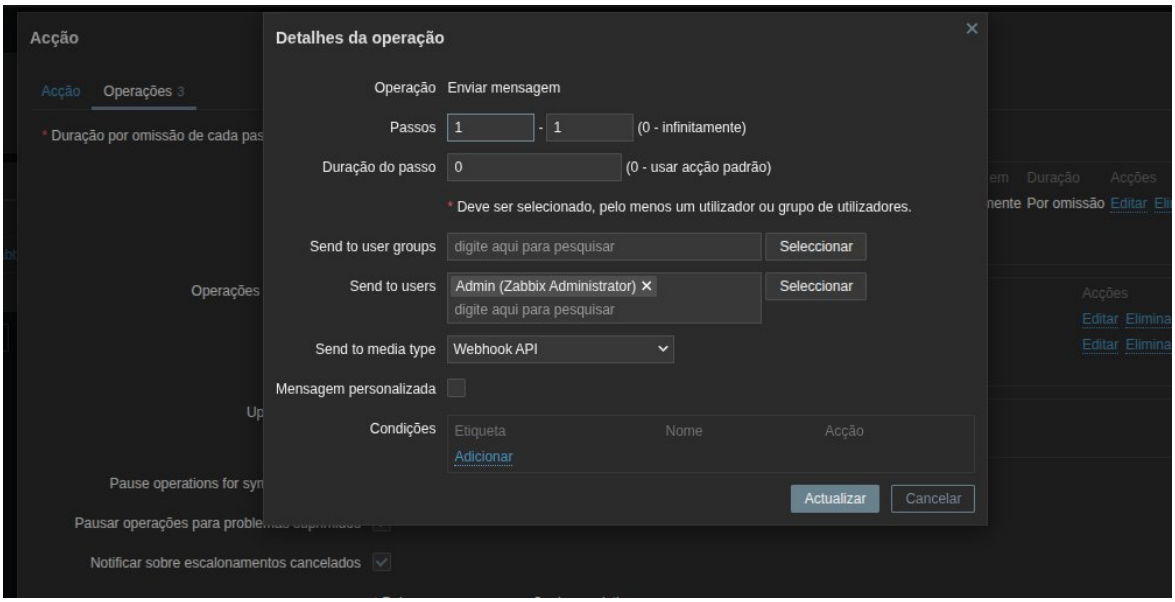
```
        value=INCIDENT_STATUS["aberto"],
        inline=False
    )

return embed


@client.event async def on_ready(): logger.info(f"Bot online como {client.user}") bot_ready.set()

async def send_discord_message(data: dict): await bot_ready.wait()

mention = "@everyone" if data.get("severity", "").lower() in ["high", "disaster"] else ""

# Gera um ID único para o alerta se não existir
if 'id' not in data:
    unique_id = hashlib.md5(f"{data.get('trigger')}{data.get('host')}{datetime.now().isoformat()}".encode()).hexdigest()[:8]
    data['id'] = unique_id

try:
    # Cria o embed com a cor apropriada
    embed = create_alert_embed(data)

    # Cria a view com o botão de claim
    view = ClaimView(data)

    channel = client.get_channel(CHANNEL_ID) or await client.fetch_channel(CHANNEL_ID)
    await channel.send(f"{mention}", embed=embed, view=view)
    logger.info(f"Alerta enviado - Trigger: {data.get('trigger')}, Host: {data.get('host')}")
except Exception as e:
    logger.error(f"Erro ao enviar alerta para Discord: {e}")


def _send_incident_sync(message: str): global id_item

headers = {
    "Authorization": f"Bearer {ZABBIX_API_TOKEN}",
    "Content-Type": "application/json-rpc"
}
body = {
    "jsonrpc": "2.0",
    "method": "history.push",
    "params": [{
        "itemid": ZABBIX_ITEM_ID,
        "value": message
    }],
    "id": id_item
}
r = requests.post(ZABBIX_URL, json=body, headers=headers, timeout=5)
logger.info(f"Resposta Zabbix: {r.text}")
r.raise_for_status()
id_item += 1


async def send_incident_to_zabbix(message: str): await asyncio.to_thread(_send_incident_sync, message)

@client.event async def on_message(message): if message.author.bot: return

if message.channel.id != CHANNEL_ID_MANUAL:
    return
```

```
if message.content.startswith("/incident"):
    incident_text = message.content.replace("/incident", "").strip()

    if not incident_text:
        await message.channel.send(" Usa: /incident <descrição>")
        return

    await send_incident_to_zabbix(incident_text)

    await message.channel.send(
        f" Incidente criado no Zabbix:\n**{incident_text}**"
    )


@app.post("/zabbix") async def zabbix_webhook(request: Request): data = await request.json()
asyncio.create_task(send_discord_message(data)) return {"ok": True}

if name == "main": loop = asyncio.get_event_loop() loop.create_task(client.start(DISCORD_TOKEN))

config = uvicorn.Config(app, host="0.0.0.0", port=8000, loop="asyncio")
server = uvicorn.Server(config)
loop.run_until_complete(server.serve())
```



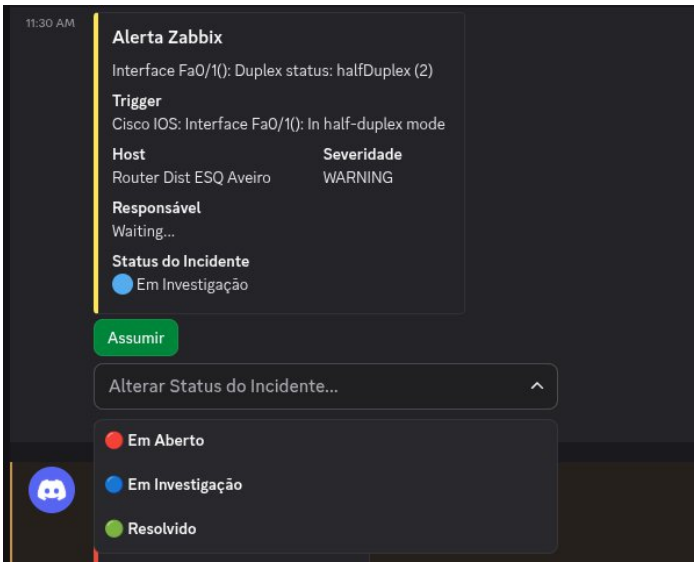


Isto define que cada mensagem vinda do discord é considerado um incidente manual de severidade alta

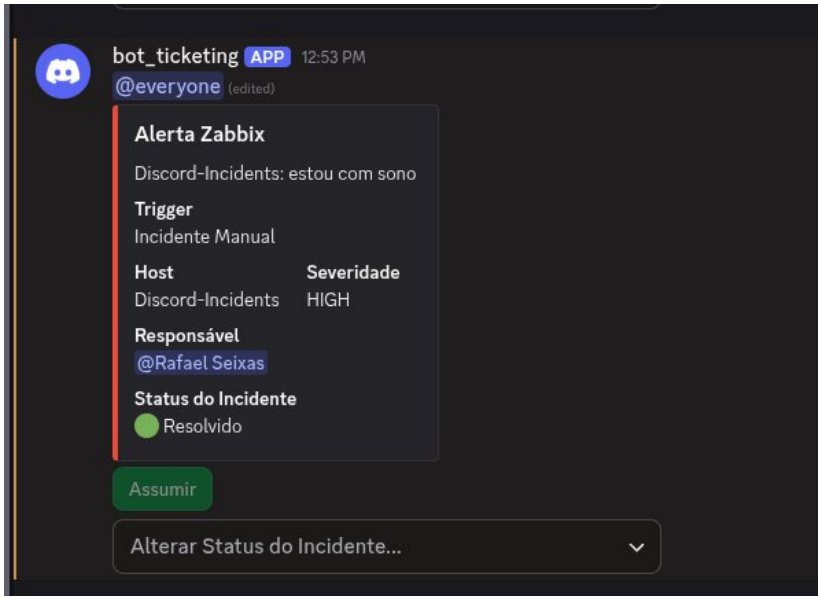


Ação que envia para o discord automaticamente qualquer trigger acima de WARNING para o Discord via WebhookAPI

Resto das configurações no ficheiro de configs



Exemplo de um Alerta no canal do discord com severidade WARNING e demonstração da box Status do Incidente



Exemplo de mensagem de Incidente Manual com Severidade HIGH, nota-se que é possivel assumir um responsável único inalterável.

Conclusão

O projeto "Moliceiros da Ria" possibilitou a criação de uma infraestrutura de rede que conecta de forma eficaz as três localidades da empresa: Águeda, Aveiro e Oliveira de Azeméis. A base da conectividade foi implementada, incluindo o encaminhamento dinâmico OSPF, túneis VPN Site-to-Site e a atribuição de serviços essenciais como DHCP e DNS, garantindo assim a comunicação entre os diferentes polos.

Um dos grandes destaques do projeto foi a implementação da camada de Gestão e Monitorização, que contou com a solução Zabbix integrada a um Bot de Discord e Dashboards no Grafana. Essa parte, junto com a automação de backups, proporciona uma visão proativa da infraestrutura em funcionamento.

Entretanto, durante a fase final de implementação, algumas limitações e funcionalidades foram identificadas e deixadas para "Trabalhos Futuros", devido à complexidade técnica e a restrições de tempo:

- Segurança Avançada (ACLs e Firewall): Embora o plano de endereçamento e as VLANs estejam operacionais, a implementação das Listas de Controlo de Acesso (ACLs) para uma filtragem mais precisa do tráfego entre VLANs não foi concluída, a tentativa de bloquear o trafego entre as vlans da mesma rede mantendo o acesso aos serviços da vlan de servidores provou-se um grande desafio, pois a falta de

especificidade não garante segurança enquanto que garanti-la torna-se um trabalho bastante intencivo ao nivel da logical , o resultado foi uma rede sem ACLs para vlans, o que resultou numa rede interna mais permissiva do que o desejado.

- Gestão Centralizada (Active Directory): A integração total de utilizadores e computadores em um domínio Active Directory ainda não foi finalizada, mantendo a gestão de identidades descentralizada ou baseada em Linux.

- Redundância de Gateway (HSRP): A alta disponibilidade no nível do gateway (HSRP) não foi implementada, o que deixa pontos únicos de falha na saída de cada rede local.

- Integração RADIUS com VoIP: O serviço FreeRADIUS foi configurado com sucesso para autenticação Wi-Fi, mas a integração específica para autenticação de dispositivos VoIP ainda não foi realizada.

Em resumo, o projeto resultou em uma rede "Core" que está operacional e altamente monitorável, atendendo aos requisitos de conectividade e serviços web/mail. As lacunas identificadas servirão como base para uma "Fase 2".